

第 24 章: Module Packages (模块包)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 前面几章讲的都是"单文件模块", 本章把它升级为"模块包" (package) ——用文件夹来组织一批模块文件。你将学到包导入、init.py、包内相对导入 (点号语法) 以及命名空间包。这是理解标准库 (如 email、tkinter、http.server) 和第三方库工作原理的关键, 也是下一章进阶模块话题 (如 name、getattr、字符串导入) 的基础。

24.1 开篇引言

英文原文

So far, when we've imported modules, we've been loading files. This represents typical module usage, and it's probably the technique you'll use for most imports you'll code, especially early in your Python career. The module import story, though, is richer than implied up to this point. This chapter extends it to present module packages—collections of module files that normally correspond to folders (a.k.a. directories) on your device. It covers four topics:- Package imports, which give part of a folder path leading to a file- Packa

ges themselves, which organize modules into folder bundles- Package-relative imports, which use dots within a package to limit search- Namespace packages, which build a package that may span multiple folders As you'll find, a package import turns a folder on your computer into another Python namespace, with attributes corresponding to the module files and subfolders that the folder contains. As you'll also learn, package imports are sometimes required to resolve ambiguity when multiple program files of the same name are installed on a device. Packages are a somewhat advanced topic, which many readers can defer until they gain experience with file-based modules. That said, packages provide an easy way to organize code files that avoids same-name conflicts and is employed by many standard-library and third-party tools that you'll be using. While packages, like much in Python, have grown convoluted over time, a basic knowledge of their usage can be beneficial to most Python learners.

中文翻译

到目前为止，我们导入模块时加载的都是文件。这代表了典型的模块用法，也很可能是你在编程生涯早期会使用最多的导入技巧。不过，模块导入的故事远比前面暗示的要丰富。本章将把它扩展为模块包（package）——即通常与设备上文件夹（也称目录）对应的模块文件集合。本章涵盖四个主题：- 包导入（Package imports），给出通向某个文件

的部分文件夹路径- 包本身 (Packages) , 把模块组织成文件夹包- 包相对导入 (Package-relative imports) , 用包内的点号来限制搜索范围- 命名空间包 (Namespace packages) , 构建可能横跨多个文件夹的包你会发现, 包导入把电脑上的一个文件夹变成了另一个 Python 命名空间, 其属性与文件夹中包含的模块文件和子文件夹一一对应。你还会学到: 当设备上安装了多个同名程序文件时, 包导入有时是解决歧义的必需手段。包是一个相对高级的主题, 许多读者可以先跳过, 等对基于文件的模块有了经验之后再回头学习。话虽如此, 包提供了一种组织代码文件的简便方式, 能避免同名冲突, 而且许多你将要使用的标准库和第三方工具都采用了这种方式。虽然包——像 Python 中许多东西一样——随时间推移变得有些复杂, 但对大多数 Python 学习者来说, 掌握基本的用法仍然是有益的。

深度理解

·核心概念: 把"导入"从"加载一个文件"扩展为"加载一个文件夹路径"。点号分隔的名字 `dir1.dir2.mod` 在 Python 世界里成为一条通往模块的路径, 文件夹目录由此获得了"命名空间"的身份。

·底层视角: 包导入最终仍然是"按名查找 + 缓存加载"。解释器先找到 `sys.path` 上的目录 `dir1`, 再沿点号路径逐层深入, 每进入一个目录都会在内存在中创建一个对应的模块对象 (namespace) , `import dir1.dir2.mod` 执行后会产生 `dir1`、`dir1.dir2`、`dir1.dir2.mod` 三个模块对象, 它们都缓存在 `sys.modules` 中。

·设计思想: 为什么不用平台相关的路径分隔符 (`\`、`/`) ?

因为导入语句要写成跨平台通用的代码——把"平台差异"全部交给 `sys.path` 配置去处理，代码本身保持中立。

·实际问题：同名冲突。两个程序都叫 `utilities.py` 时，只有包（目录名作为前缀）能让引用唯一化。

·初学者误区：以为 `import dir1.dir2.mod` 里的目录是"工作目录下的路径"。其实 `dir1` 必须位于 `sys.path` 的某个条目中，且导入路径不能使用 `C:\`、`/Users/me` 这类平台语法——把平台路径放进 `sys.path`，把纯点号路径放进代码。

24.2 使用包 (Using Packages)

英文原文

To load items in a package folder, you'll use the normal import statements and tools we've already met, but provide a path of names that reflects a path of nested folders. Let's get started with this user-guide side of the packages story.

中文翻译

要加载包文件夹中的内容，你仍使用我们已经熟悉的普通 `import` 语句和工具，只是要提供一条由名字组成的路径，这条路径对应着嵌套的文件夹路径。让我们从"用户手册"这一侧开始讲包的故事。

深度理解

- 核心概念：包的使用者视角——"用点号代替斜杠"。目录嵌套被翻译成点号分隔的名字路径。
- 底层实现：点号路径在解释器内部就是名字分层查找的过程，每层对应 `sys.modules` 中的一个缓存条目。
- 设计原因：保持语句形态不变（还是 `import ... / from ... import ...`），只是参数形态从单名变点号路径，学习成本最低。
- 实际问题：无需记住模块在磁盘的绝对位置，代码天然可移植。
- 初学者误区：认为目录必须手动加入 `sys.path` 才可用——其实 REPL 的当前目录（"home"目录）自动在搜索路径上，只要当前目录是包的容器即可。

24.3 包导入 (Package Imports)

英文原文

Coding package imports is straightforward. In all the places where you have been naming a simple file in your import operations, you can instead list a path of names separated by periods. For instance, this works in an import statement:

```
python import dir1.dir2.mod
```

The same goes for from statements:

python from dir1.dir2.mod import var And this also applies to already-imported items in reload calls:

```
python from importlib import reload reload(dir1.dir2.mod)
```

The "dotted path" in these contexts is assumed to correspond to a path through the folder hierarchy on your device, leading to the module file mod.py, or other component with basename mod (as we've seen, it might also be a bytecode file, C extension module, or other). Most importantly here, the preceding paths indicate that on your device there is a directory dir1, which has a subdirectory dir2, which contains a module file mod.py (or similar). Furthermore, these paths imply that dir1 resides within some container directory dir0, which is a part of the normal module search path. In other words, these imports and reloads imply a directory structure that looks something like this on Unix and Windows, respectively:

```
dir0/dir1/dir2/mod.py # Or mod.pyc, mod.so, etc. dir0\dir1\dir2\mod.py # Ditto, on Windows
```

The container directory dir0 needs to be added to your module search path unless it's an automatic part of that path, exactly as if dir1 were a simple module file. More formally, the leftmost component in a package import path is relative to (located in) a directory included in the sys.path module search-path list we explored in Chapter 22. From there down, though, the

import statements in your script explicitly give the directory paths leading to modules in packages. The dotted-path syntax of packages is platform neutral, but also reflects the fact that folder paths in import statements become nested objects: `dir1.dir2.mod` traverses three module objects after an import. This syntax also explains why Python complains about a file not being a package if you forget to omit the `.py` extension in import statements: it's taken to mean a package import!

中文翻译

编写包导入非常简单。在你过去写 `import` 操作、命名一个简单文件的所有地方，现在都可以改为列出由句点分隔的名字路径。例如，以下写法在 `import` 语句中有效：`python import dir1.dir2.mod` 从语句也一样：`python from dir1.dir2.mod import var` 这同样适用于对已导入对象的 `reload` 调用：`python from importlib import reload reload(dir1.dir2.mod)` 在这些场景中，“点号路径”被假定为与设备上的文件夹层级相对应，通向模块文件 `mod.py`，或其他以 `mod` 为基名的组件（正如我们所见，它也可能是字节码文件、C 扩展模块等）。这里最重要的是：前面的路径表明你的设备上存在一个目录 `dir1`，它有一个子目录 `dir2`，`dir2` 中包含模块文件 `mod.py`（或类似物）。此外，这些路径还隐含：`dir1` 位于某个容器目录 `dir0` 之内，而 `dir0` 是正常模块搜索路径的一部分。换句话说，这些导入和 `reload` 隐含的目录结构在 Unix 和 Windows 上分别如下：`dir0/dir1/dir2/mod.py` # 或者是 `mod.pyc`、`mod.so` 等 `dir0\dir1\dir2\mod.py`

ir1\dir2\mod.py # Windows 上同理容器目录 dir0 需要被加入你的模块搜索路径——除非它本来就自动属于该路径——这与你把 dir1 当作简单模块文件处理完全一样。更正式地说，包导入路径中最左边的组件是相对于（位于）sys.path 模块搜索路径列表中的某个目录的，这一点我们在第 22 章讨论过。但从该目录往下，你脚本中的 import 语句就显式地给出了通向包内各模块的目录路径。包的点号语法是平台无关的，但它也反映了一个事实：import 语句中的文件夹路径会变成嵌套对象——执行一次导入后，dir1.dir2.mod 会穿越三个模块对象。这种语法也解释了为什么如果你忘记在 import 语句中去掉 .py 扩展名，Python 会抱怨“该文件不是一个包”（"not a package"）：因为带扩展名的写法被理解为包导入！

深度理解

- 核心概念：包导入 = 用点号路径代替单文件名。点号路径的最左组件必须位于 sys.path 的某个条目中，其余部分由代码显式给出。
- 底层实现：import dir1.dir2.mod 执行时，解释器（导入系统）会递归地：在 sys.path 中找到 dir1（导入并缓存为模块对象）→ 在 dir1.path 中找 dir2 → 在 dir2.path 中找 mod。sys.modules 里出现三个键：'dir1'、'dir1.dir2'、'dir1.dir2.mod'。最后一条语句 import dir1.dir2.mod 只是把最内层模块绑定到名字 dir1 上——所以必须通过 dir1.dir2.mod.var 才能访问属性。
- 设计原因：. 是合法标识符字符，用点号既能写进语句、又天然跨平台，还能表达嵌套关系。相比之下 C:\ 或 / 根

本不是合法 Python 标识符语法。

·实际问题：忘记去掉 .py 扩展名时，import mod.py 被解释成"导入名为 mod.py 的包"，于是报 ModuleNotFoundError: No module named 'mod.py'; 'mod' is not a package——这一报错信息（not a package）正是本章主题的体现。

·初学者误区：以为点号路径能像变量一样动态拼接。路径是字面量，不是变量值——from dir2 import mod 不会因为前面执行过 from dir1 import dir2 就去找 dir1 下的 dir2（后文有完整演示）。

代码分析

```
import dir1.dir2.mod          # dir1dir1.dir2dir1.dir2...
from dir1.dir2.mod import var # var var
from importlib import reload
reload(dir1.dir2.mod)         #
```

解释器如何处理：执行第一行时，Python 的导入机制（importlib）先检查 sys.modules 缓存；未命中则沿 sys.path 找到 dir0（容器），在其中发现目录 dir1，创建模块对象并填入缓存；随后以 dir1 的 path 为搜索集继续找 dir2，再在 dir2.path 中找到 mod.py 并编译执行其顶层代码。整个过程只有最后一个模块会"运行代码"，前两层的文件夹模块只是建立命名空间骨架。from ... import var 则是在此基础上把 var 这个名字复制到当前作用域。reload 只重载路径最右端的单个模块——注意上面的输出中没有重跑 dir1 的初始化代码。

24.4 包与模块搜索路径 (Packages and the Module Search Path)

英文原文

If you use this feature, keep in mind that the directory paths in your import statements can be only variables separated by periods. You cannot use any platform-specific path syntax in your import statements, such as `C:\dir1`, `/Users/me/dir1`, or `../dir1`—these do not work syntactically. Instead, use any such platform-specific syntax in your module search path settings to name the directories containing your packages. For instance, in the prior example, `dir0`—the directory name you add to your module search path—can be an arbitrarily long and platform-specific directory path leading up to `dir1`. You cannot use an invalid statement like this: `python import C:\Users\me\mycode\dir1\dir2\mod` # Error: illegal syntax (Windows) `import /Users/me/mycode/dir1/dir2/mod` # Ditto, on Unix But you can add a path like `C:\Users\me\mycode` or `/Users/me/mycode` to either your `PYTHONPATH` environment variable, a strategically placed `.pth` file, or `sys.path` itself in manual code, and then say this in your script: `python import dir1.dir2.mod` # OK: variables and periods In effect, entries on the module search path provide platform-specific directory prefixes, which lead to the leftmost names of package paths in import and from

statements. These import statements themselves provide the remainder of the directory path in a platform-neutral fashion. As for simple file imports, you do not need to add the container directory to your module search path if it's already there. Per Chapter 22, the search path automatically includes the "home" directory (where you're working in a REPL, or the container of a launched program's top-level file), along with the standard library's containers, and the site-packages third-party install root. While none of these components require search-path mods, your module search path must include all the directories containing leftmost components in your code's package-import statements.

中文翻译

如果使用这一特性，请记住：import 语句中的目录路径只能是由句点分隔的变量。你不能在 import 语句中使用任何平台特定的路径语法，例如 C:\dir1、/Users/me/dir1 或 ../dir1——这些在语法上都不合法。相反，请在模块搜索路径设置中使用这类平台特定语法，来指定包含你的包的目录。例如，在前面的例子中，dir0——你加入模块搜索路径的目录名——可以是一段任意长的、平台特定的目录路径，一直通向 dir1。你不能使用下面这种非法语句：python import C:\Users\me\mycode\dir1\dir2\mod # 错误：非法语法 (Windows) import /Users/me/mycode/dir1/dir2/mod # Unix 上同理但你完全可以把 C:\Users\me\mycode 或 /Users/me/mycode 这样的路径加入你的 PYTHON

NPATH 环境变量、一个位置得当的 .pth 文件，或通过代码手动加入 sys.path 本身，然后在脚本中这样写：python import dir1.dir2.mod # 正确：变量和句点实际上，模块搜索路径上的条目提供了平台特定的目录前缀，通向 import 和 from 语句中包路径最左边的名字；而这些 import 语句本身则以平台无关的方式提供目录路径的其余部分。与简单的文件导入一样，如果容器目录已经在模块搜索路径上，你就不需要再添加它。按照第 22 章所述，搜索路径自动包含"home"目录（REPL 中即你当前的工作目录，或所启动程序的顶层文件所在目录）、标准库的容器目录，以及 site-packages 第三方安装根目录。虽然这些组件都不需要修改搜索路径，但你的模块搜索路径必须包含代码中包导入语句所有最左组件所在的目录。

深度理解

·核心概念：职责分离——sys.path 负责"平台相关的部分"（从盘符/根目录到包根），代码里的点号路径负责"平台无关的部分"（包根以下的嵌套）。

·底层实现：sys.path 是一个字符串列表，导入系统按顺序（从左到右）把每个条目当作查找根。对 dir1.dir2.mod 而言，解释器依次用 sys.path 的每个条目拼接 dir1 试探（<条目>/dir1 是否存在），首个命中者胜出。

·设计原因：Windows 用 \、Unix 用 /，若代码直接写平台路径，同一段代码无法在两平台运行。把平台差异隔离到环境变量（PYTHONPATH）、配置文件（.pth）或启动代码中，是 Python 可移植性的核心策略。

·实际问题：运行 `python mycode/main.py` 时，`"home"`（`mycode` 目录）自动在搜索路径上——因此脚本能导入同目录模块而无需任何配置；只有跨目录导入才需要配置。

·初学者误区：试图在 `import` 里写绝对路径或 `..`；认为修改了当前目录（`os.chdir`）就能影响导入——导入搜索依据的是 `sys.path`，不是 CWD。

代码分析

```
# import
import C:\Users\me\mycode\dir1\dir2\mod      # SyntaxError
import /Users/me/mycode/dir1/dir2/mod        # SyntaxError

#
# Windows:  set PYTHONPATH=C:\Users\me\mycode
# Unix:      export PYTHONPATH=/Users/me/mycode
import dir1.dir2.mod                          # sys.path ...
```

解释器如何处理：执行 `import dir1.dir2.mod` 时，导入机制对 `sys.path` 的每个条目执行“路径拼接 + 文件系统探查”（`importlib.machinery.PathFinder`）：把条目与 `dir1` 拼接，检查是目录（且有 `init.py`）、普通模块文件还是别的；找到就返回 loader 并加载。`.pth` 文件则是 `site` 模块启动时读取的、每行一个目录的纯文本文件，其内容会自动追加进 `sys.path`——`PYTHONPATH`、`.pth`、`sys.path` 手动追加，三者效果等价，只是生效时机不同。

24.5 创建包 (Creating Packages)

英文原文

To make packages of your own, you'll bundle module files and nested folders in a package folder. A package folder can also optionally contain an `init.py` file run on first import, and a `main.py` file run when the entire package folder is run as a bundle. The following sections demo what this looks like in code, one step at a time.

中文翻译

要创建自己的包，你需要把模块文件和嵌套文件夹打包在一个包文件夹中。包文件夹还可以（可选地）包含一个首次导入时运行的 `init.py` 文件，以及一个把整个包文件夹当作整体运行时执行的 `main.py` 文件。接下来各节将分步骤演示这在代码中是什么样子。

深度理解

- 核心概念：包 = 文件夹 + 约定的特殊文件（`init.py` 管"导入"，`main.py` 管"运行"）。
- 底层实现：两个特殊文件名都是导入系统/启动机制的"约定钩子"（hook），由解释器在特定时机自动执行，而不是由你手动调用。
- 设计原因：把"包作为库被导入"与"包作为程序被运行"两种用法分开，各用各的文件，互不干扰。

- 实际问题：一个工具包既要 import 进来复用，又要能 python 包目录直接跑，main.py 就是为此设计的。
- 初学者误区：以为 init.py 必须非空——其实空文件也合法，它还可以只用来“声明这是个包”。

24.6 基本包结构 (Basic Package Structure)

英文原文

In their simplest form, packages are just folders containing normal module files, and perhaps nested subfolders of the same. Let's set one up as a demo. The following uses indentation to represent the folder nesting we'll be using in this and the following sections:

```
dir0/ # Container listed on module search path
dir1/ # Package root folder
  mod.py # Module file
dir1.dir2/ # Nested package folder
  dir1.dir2.mod.py # Module
```

These nested folders can be created in file explorers, or with the following commands on most platforms; use backslashes on Windows, or simply use this Chapter's folder in the examples package, which has prebuilt the structure:

```
$ mkdir dir1 $ mkdir dir1/dir2 # Use backward slashes on Windows
Admin note: the names of package folders, like those of simple module files, must follow t
```

he rules for variable names because they become variables where imported. See Chapter 11 for a refresher on the constraints, but in short, use letters, digits, and underscores, and avoid reserved words.

Now, add the nested module files listed in Examples 24-1 and 24-2. Their top-level code runs on first import and creates attributes as usual, and their titles give their paths (using just Unix forward-slash separators for brevity).

Example 24-1. dir1/mod.py

```
python var = 'hack'print('Loading dir1.mod') Example 24-2. dir1/dir2/mod.py
```

```
python var = 'code'print('Loading dir1.dir2.mod')
```

中文翻译

最简单形式的包，就是包含普通模块文件（或许还有同类的嵌套子文件夹）的文件夹。让我们搭一个来演示。下面用缩进表示文件夹嵌套关系，本章及后续各节都将使用这种表示：

- dir0/ # 列在模块搜索路径上的容器
- dir1/ # 包根文件夹
- dir1 mod.py # 包内模块文件
- dir1.mod dir2/ # 嵌套包文件夹
- dir1.dir2 mod.py # 包内模块
- dir1.dir2.mod 这些嵌套文件夹可以用文件资源管理器创建，也可以在最常见平台上用以下命令创建；Windows 上请用反斜杠，或者直接使用本书示例包中本章的文件夹（已预建好该结构）：

```
$ mkdir dir1 $ mkdir dir1/dir2 # Windows 上用反斜杠
```

管理员提示：包文件夹的名字，和简单模块文件的名字一样，必须遵循变量名的规则，因为导入后它们会变成变量。约束条

件请回顾第 11 章，简单说：只用字母、数字和下划线，并避免使用保留字。现在，添加示例 24-1 和 24-2 中列出的嵌套模块文件。它们的顶层代码会在首次导入时运行并照常创建属性，标题给出了它们的路径（为简洁起见只用了 Unix 的正斜杠分隔符）。示例 24-1. `dir1/mod.py` `python var = 'hack' print('Loading dir1.mod')` 示例 24-2. `dir1/dir2/mod.py` `python var = 'code' print('Loading dir1.dir2.mod')`

深度理解

- 核心概念：目录结构直接映射为点号路径——`dir1/dir2/mod.py` 对应 `dir1.dir2.mod`。这就是“包是文件夹、文件夹是命名空间”的最直观体现。
- 底层实现：导入 `dir1.mod` 时，解释器在 `sys.path` 的条目（本例为 REPL 当前目录 `dir0`）下找到 `dir1` 目录，再在 `dir1` 内找到 `mod.py`，编译执行其顶层代码。`print('Loading dir1.mod')` 输出的时机即“首次导入时刻”。
- 设计原因：包名必须满足变量名规则，是因为 `import dir1` 后 `dir1` 就成了当前命名空间里的一个变量；连字符、空格、以数字开头都会导致 `import` 语句无法解析。
- 实际问题：`mkdir` 只创建文件夹；`init.py` 未加之前，`dir1` 已经是“命名空间包”（无初始化文件），但后文会说明它只是功能稍弱的形态。
- 初学者误区：以为包名可以与模块名不同规则；或以为目录结构必须手动“注册”才能导入——只要容器在 `sys.path` 上即可。

代码分析

```
# dir1/mod.py — 24-1
var = 'hack'                # dir1.mod var
print('Loading dir1.mod') #

# dir1/dir2/mod.py — 24-2
var = 'code'                # dir1/mod.py var
print('Loading dir1.dir2.mod')
```

解释器如何处理：import dir1.mod 时，Python 为 mod.py 创建模块对象、设置 name = 'dir1.mod'、file 指向磁盘路径，然后逐条执行顶层语句：赋值 var = 'hack' 写入模块的全局命名空间（即成为 dir1.mod.var），print 输出一行。同一进程内再次 import 不会重跑——模块已缓存于 sys.modules，直接返回缓存对象。两个 mod.py 各自的 var 属于各自模块命名空间，互不覆盖。

24.7 使用基础包 (Using the Basic Package)

英文原文

To use our module package, import its modules from a REPL or another file just as you would for a simple.py file, but use dots to specify the path below the dir0 folder in the search path—which is the current directory in a REPL:

```
$ python3 >> import dir1.mod # Package paths in i
import Loading dir1.mod >> dir1.mod.var # Module c
```

```
ode run on import'hack'>> import dir1.dir2.mod # A
further-nested path Loading dir1.dir2.mod >> dir1.dir2.mod.var # Repeat path to get item at end'code'
```

As for simple top-level modules, the code of module nested in a package is run only when first imported:

```
>> import dir1.mod # No-op if already imported >>
import dir1.dir2.mod
```

And you generally must import a nested module in order to use its attributes—because modules corresponding to folders don't go back to the filesystem on attribute fetches, it's not enough to import just a root folder:

```
$ python3 >> import dir1 # Gets dir1, nothing below it >> dir1.dir2.mod.var
AttributeError: module'dir1' has no attribute'dir2'
```

```
$ python3 >> import dir1.dir2 >> dir1.dir2.mod.var
AttributeError: module'dir1.dir2' has no attribute'mod'
```

```
$ python3 >> import dir1.dir2.mod # List full path to target Loading dir1.dir2.mod >> dir1.dir2.mod.var'code'
```

All this works the same in the `from` statement—which, as we've seen, is really just `import` with an extra copy of names. Again, though, we must list a folder by its path in an `import` statement to be able to access its contents, and the paths listed in imports are taken l

iterally, not fetched from variables:

```
$ python3 >> from dir1.mod import var # Package paths in from Loading dir1.mod >> var # Don't repeat path to item'hack'>> from dir1.dir2.mod import var Loading dir1.dir2.mod >> var'code'>> from dir1 import dir2 # Paths taken literally >> from dir2 import mod # Not from variable values ModuleNotFoundError: No module named'dir2'
```

One potential advantage of `from` here is that it avoids repeating a package path every time an item in a package is used. With `import`, you generally must repeat the full path each time, but `from` lets you code the path just once, and use the simple name fetched from it everywhere; this is especially useful if the package's directory structure later changes (as software is wont to do):

```
>> import dir1.dir2.mod # import requires paths >> dir1.dir2.mod.var'code'>> from dir1.dir2.mod import var # from can shorten paths >> var'code'>> from dir1.dir2 import mod # And avoids repeating them >> mod.var'code'>> import dir1.dir2.mod as mod # Though "as" can too >> mod.var'code'
```

As the last example shows, though, the `as` extension introduced in the preceding chapter can shorten up paths the same way as `from`—as can a simple manual reassignment, which the `as` sugarcoats (more on `as` in the next chapter).

中文翻译

要使用我们的模块包，从 REPL 或其他文件中导入其模块，方式与导入简单 .py 文件完全一样，只是要用点号来指定搜索路径上 dir0 之下的路径——在 REPL 中，dir0 就是当前目录：

```
$ python3 >>> import dir1.mod # import 中的包路径Loading dir1.mod >>> dir1.mod.var # 模块代码在导入时运行'hack'>>> import dir1.dir2.mod # 更深的嵌套路径Loading dir1.dir2.mod >>> dir1.dir2.mod.var # 要重复完整路径才能取到末端的条目'code'与简单的顶层模块一样，包内嵌套模块的代码只在首次导入时运行：>>> import dir1.mod # 已导入过，此句无操作>>> import dir1.dir2.mod 而且你通常必须导入嵌套模块才能使用它的属性——因为对应文件夹的模块不会在属性访问时重新回到文件系统查找，所以仅仅导入根文件夹是不够的：$ python3 >>> import dir1 # 只得到 dir1，它下面的内容什么都没有>>> dir1.dir2.mod.var AttributeError: module 'dir1' has no attribute 'dir2'$ python3 >>> import dir1.dir2 >>> dir1.dir2.mod.var AttributeError: module 'dir1.dir2' has no attribute 'mod'$ python3 >>> import dir1.dir2.mod # 列出通向目标的完整路径Loading dir1.dir2.mod >>> dir1.dir2.mod.var'code'这一切在 from 语句中同样成立——正如我们所见，from 其实只是 import 外加一次名字复制。但同样地，我们必须在一个 import 语句中按路径列出文件夹才能访问其内容，而且 import 中列出的路径是字面量，不会从变量中取值：$ python3 >>> from dir1.mod import var # from 中的包路径Loading dir1.mod >>> var # 无需重复路径即可使用条目'hack'
```

```
>>> from dir1.dir2.mod import var Loading dir1.dir2.
mod >>> var'code'>>> from dir1 import dir2 # 路径
按字面量处理>>> from dir2 import mod # 不会从变量
值中取路径ModuleNotFoundError: No module named'
dir2'这里 from 的一个潜在优势是：它避免了每次使用包内
条目时都要重复包路径。使用 import 时，你通常每次都必
须重复完整路径；而 from 让你只写一次路径，之后到处使
用取到的简单名字。当包目录结构日后发生变化（软件常如
此）时，这一优势尤其有用：>>> import dir1.dir2.mod
# import 需要完整路径>>> dir1.dir2.mod.var'code'>>
> from dir1.dir2.mod import var # from 可以缩短路径
>>> var'code'>>> from dir1.dir2 import mod # 并且
避免重复它们>>> mod.var'code'>>> import dir1.dir2.
mod as mod # 不过 "as" 也能做到>>> mod.var'code'
不过，正如最后一个例子所示，前一章引入的 as 扩展可以
像 from 一样缩短路径——简单的手工重新赋值也能做到，
as 只是它的语法糖（as 的更多内容见下一章）。
```

深度理解

·核心概念：属性不会自动触发导入。import dir1 之后 dir1.dir2 并不存在，因为文件夹模块对象不会“顺手”去磁盘扫描子目录——属性查找只是查模块对象的 dict。

·底层实现：import dir1 时，解释器创建 dir1 模块对象，但不会预加载 dir2、mod——它们是独立的缓存条目，只有各自的 import 语句执行后才会出现。属性访问 dir1.dir2 就是一次普通字典查找，找不到就抛 AttributeError。

- 设计原因：如果每次属性访问都触发文件系统 I/O，程序会慢得不可接受；且"按需导入" (lazy) 让导入开销精确可控。文件夹模块在内存中只是"命名空间的占位容器"。
- 实际问题：from dir1 import dir2 成功（把 dir2 模块对象绑定为一个名字）但 from dir2 import mod 失败——路径是字面量，解释器不会因为你"手里有 dir2 这个对象"就把它当查找起点。这正是"路径字面量"与"变量"的本质区别。
- 初学者误区：以为 import dir1 就能用 dir1.dir2.mod；以为 from 会改变后续 import 的查找上下文。两者的底层都只是"加载 + 名字绑定"，查找根永远是 sys.path（或当前包）。

代码分析

```
import dir1                # dir1 dir2 mod
dir1.dir2.mod.var          # dir1 dir2
# AttributeError: module 'dir1' has no attribute 'dir2'

import dir1.dir2.mod        # dir2mod
dir1.dir2.mod.var          # 'code'

from dir1 import dir2       # dir2
from dir2 import mod        # dir2 sys.path
# ModuleNotFoundError: No module named 'dir2'

import dir1.dir2.mod as m   # as m = dir1.dir2.mod
m.var                      # 'code'
```

解释器如何处理：每次 import 都走同一套流程——查 sys.modules 缓存 → 未命中则查找并加载 → 绑定名字。from dir1 import dir2 时，解释器加载 dir1 后从它的属性

字典中取出 `dir2`（此时 `dir1.dir2` 因为此前被加载过而存在）复制到当前作用域；而 `from dir2 import mod` 的查找根是 `sys.path`，顶层根本没有 `dir2` 目录，于是报错。`import ... as m` 的实质是 `m = dir1.dir2.mod` 的语法糖：解释器加载完模块后，把模块对象绑定到名字 `m` 而非全路径。

24.8 包的 `init.py` 文件 (Package `init.py` Files)

英文原文

If you need to run setup code when a package folder is imported, put it in a file named `init.py` and store it in the package's folder. Python will automatically run this file's code the first time the package is imported during a program run (or REPL session). Here's how this augments our package's structure:

```
dir0/ dir1/ init.py # Run on first import of dir1 module
dir2/ init.py # Run on first import of dir1.dir2 module
```

The new `init.py` files are listed in Examples 24-3 and 24-4.

Example 24-3. `dir1/init.py`

```
python var = 'Python' print('Running dir1.init.py')
Example 24-4. dir1/dir2/init.py
```



```
python var = 3.12 print('Running dir1.dir2.init.py')
```

中文翻译

如果你需要在导入包文件夹时运行一些初始化代码，请把它放进名为 `init.py` 的文件中，并存储在包的文件夹里。Python 会在一次程序运行（或 REPL 会话）中首次导入该包时自动运行这个文件的代码。下面是这给我们的包结构带来的变化：

- `dir0/ dir1/ init.py` # 首次导入 `dir1` 时运行 `mod.py`
- `dir2/ init.py` # 首次导入 `dir1.dir2` 时运行 `mod.py`

新的 `init.py` 文件列在示例 24-3 和 24-4 中。示例 24-3. `dir1/init.py`

```
python var = 'Python' print('Running dir1.init.py')
```

示例 24-4. `dir1/dir2/init.py`

```
python var = 3.12 print('Running dir1.dir2.init.py')
```

深度理解

- 核心概念：`init.py` 是“包级模块”——它没有磁盘上的独立模块身份，但 Python 会为每个包目录创建一个模块对象，并把这个文件的代码作为该对象的顶层代码执行。
- 底层实现：导入 `dir1.dir2.mod` 时，解释器依次进入 `dir1`、`dir2` 两个目录，每进入一个就发现并执行其 `init.py`，把它们编译为模块对象（`dir1`、`dir1.dir2`），最后才执行 `mod.py`。因此打印顺序为：`Running dir1.init.py` → `Running dir1.dir2.init.py` → `Loading dir1.dir2.mod`。
- 设计原因：文件夹本身没有“代码文件”承载初始化逻辑，`init.py` 就是给这个虚拟模块提供代码入口的约定文件。名字中的“init”正暗示其初始化钩子的身份。

·实际问题：包需要建立数据库连接、加载配置、创建数据文件、聚合导出名字等，init.py 就是唯一可靠的在"包首次被使用"时执行这些操作的地方。

·初学者误区：以为 init.py 会被多次执行——只有首次导入（或通过该目录）时执行一次，之后进 sys.modules 缓存；也不要把 init.py 当成可以直接运行的脚本（那是 main.py 的职责）。

代码分析

```
# dir1/__init__.py — 24-3
var = 'Python'                # dir1.dir1.var
print('Running dir1.__init__.py')

# dir1/dir2/__init__.py — 24-4
var = 3.12                    # dir1.dir2.var
print('Running dir1.dir2.__init__.py')
```

解释器如何处理：执行 import dir1.dir2.mod 时，导入机制把 dir1 目录识别为包（发现 init.py），创建模块对象 dir1 并执行其 init.py；接着对 dir2 重复同样流程；最后加载 mod.py。三个文件各执行一遍顶层代码，var 分别在三个不同命名空间里各有一个版本：dir1.var ('Python')、dir1.dir2.var (3.12)、dir1.dir2.mod.var ('code')——互不冲突，这正是命名空间隔离的体现。

24.9 使用更新后的包 (Using the Updated Package)

英文原文

These special `init.py` files are optional in each folder of a package. Because they are run on the first import of or through a folder level, though, they provide a natural hook for kicking off package-specific initializations (hence their abbreviated names). In fact, their assignments serve to initialize the namespace that corresponds to a folder on your device—as for files, they create attributes of the package's module object:

```
$ python3 >> import dir1.dir2.mod # Runs all init.py files
Running dir1.init.py Running dir1.dir2.init.py Loading dir1.dir2.mod
>> dir1.var # Name in init.py's namespace'Python'>> dir1.dir2.var 3.12
>> dir1.dir2.mod.var # Name in nested mod.py namespace'code'
```

Technically speaking, packages with `init.py` files are called "regular" packages, and those without them are called "namespace" packages, and the demo was a single-folder "namespace" package until adding `init.py` made it "regular." We'll get into the details behind this distinction later, but apart from the fact that "regular" packages have precedence during module search, the difference is mostly a historical artifact today, and won't matter in most roles.

Of course, `init.py` files can also do more than print beans; we'll also explore their roles later in this chapter.

As noted earlier, reload works with package paths too, if you've already imported the paths in question. Continuing the prior REPL session:

```
>> from importlib import reload >> reload(dir1) Running dir1.init.py <module'dir1' from'../LP6E/Chapter24/dir1/init.py'> >> reload(dir1.dir2) Running dir1.dir2.init.py <module'dir1.dir2' from'../LP6E/Chapter24/dir1/dir2/init.py'> >> reload(dir1.dir2.mod) Loading dir1.dir2.mod <module'dir1.dir2.mod' from'../LP6E/Chapter24/dir1/dir2/mod.py'>
```

Notice from the initialization prints that this call reloads just the single module on the right end of the path. To do better, we'll code a reloader tool in the next chapter that, given a package root, can load all the items on a package path like this, one at a time; See "Example: Transitive Module Reloads".

中文翻译

这些特殊的 init.py 文件在包的每个文件夹中都是可选的。不过，由于它们会在“首次导入该目录、或经由该目录导入”时运行，它们为启动包特定的初始化工作提供了天然的钩子（名字因此缩写为 init）。事实上，它们中的赋值语句正是用来初始化设备上与文件夹对应的命名空间——与文件模块一样，它们为包的模块对象创建属性：\$ python3 >>> import dir1.dir2.mod # 运行沿途所有的 init.py 文件Running dir1.init.py Running dir1.dir2.init.py Loading dir1.dir2.mod >>> dir1.var # init.py 命名空间中的名字'Python'>>> dir1.dir2.var 3.12 >>> dir1.dir2.mod.var #

嵌套 `mod.py` 命名空间中的名字 `'code'` 严格来说，带 `init.py` 文件的包被称为“常规”包（regular packages），不带它们的被称为“命名空间”包（namespace packages）；本演示在添加 `init.py` 之前，其实就是一个单文件夹的“命名空间”包。我们稍后会深入讲这个区分的细节，但除了“常规”包在模块搜索中具有优先权之外，这种差异在今天主要是历史遗留，在大多数场景下无关紧要。当然，`init.py` 文件能做的远不止打印标记；本章稍后我们还会探讨它们的其他作用。如前所述，只要相关路径已经导入过，`reload` 对包路径同样有效。延续之前的 REPL 会话：

```
>>> from importlib import reload
>>> reload(dir1)
Running dir1.init.py
<module 'dir1' from '/.../LP6E/Chapter24/dir1/init.py'>
>>> reload(dir1.dir2)
Running dir1.dir2.init.py
<module 'dir1.dir2' from '/.../LP6E/Chapter24/dir1/dir2/init.py'>
>>> reload(dir1.dir2.mod)
Loading dir1.dir2.mod
<module 'dir1.dir2.mod' from '/.../LP6E/Chapter24/dir1/dir2/mod.py'>
```

注意这些初始化打印：该调用只重载路径最右端的单个模块。想要做得更好，我们会在下一章编写一个重载工具，给它一个包根，就能像这样把包路径上的所有条目逐个加载一遍；参见“示例：传递式模块重载”（Example: Transitive Module Reloads）。

深度理解

- 核心概念：“常规包 vs 命名空间包”的分界线就是有没有 `init.py`。这个区别今天主要影响搜索优先级，而非功能。
- 底层实现：`reload()` 拿到模块对象后，重新编译并执行其源码，然后原地更新模块命名空间（dict 被清空重填，但模块对象本身不换）。包目录模块的重载就是重新执行它

的 `init.py`；最右端模块的重载就是重新执行 `mod.py`。注意 `reload` 不会递归重载依赖。

·设计原因：重载只针对“一个模块对象”，而一条包路径含多个模块对象，所以只重载末端。递归重载留给开发者自己写工具（下一章实现），保持核心机制简单。

·实际问题：交互式开发时改了包内代码，`reload(dir1.dir2.mod)` 能立即生效；但要刷新 `dir1` 的 `init.py`，就得单独 `reload(dir1)`——两者是独立操作。

·初学者误区：以为 `reload` 会刷新所有下层模块；以为 `reload` 后其他模块对该模块名字的引用会自动更新——重载只改模块内部命名空间，引用它的变量指向的是同一个模块对象，通常会“看到”新内容，但 `from x import var` 复制出来的旧名字不会更新。

代码分析

```
from importlib import reload
reload(dir1)           # dir1/__init__.py dir1
reload(dir1.dir2)      # dir1/dir2/__init__.py
reload(dir1.dir2.mod)  # mod.py —
```

解释器如何处理：`reload()` 依次执行：从 `sys.modules` 找到模块对象 → 重新编译源码 → 在受限全局命名空间（模块的 `dict`）中执行新字节码，同时保留 `name`、`file` 等特殊属性 → 返回模块对象。由于每次只处理一个模块对象，`import dir1.dir2.mod` 产生的三个对象必须调用三次 `reload` 才能全部刷新；而且 `reload` 后，代码里其他模块通过 `dir1.dir2.mod.var` 这类引用读取的属性会立刻取到新值（因为是同一对象）。

24.10 包的 main.py 文件 (Package main.py Files)

英文原文

Finally, if you want a package's users to be able to run the package as though it were a program or script, add main.py files to each folder where you wish to support this mode; Python will automatically run these files each time their container folder is launched like a program. Here's how this last bit augments our package's structure:

```
dir0/ dir1/ main.py # Run whenever dir1 bundle is run
init.py mod.py dir2/ main.py # Run whenever dir1.dir2
bundle is run init.py mod.py
```

The added main.py files are listed in Examples 24-5 and 24-6; they're simple so we can focus on packages.

Example 24-5. dir1/main.py

```
python print('Executing dir1.main.py')
```

Example 24-6. dir1/dir2/main.py

```
python print('Executing dir1.dir2.main.py')
```

When present, main.py files are automatically run for a variety of launch options, including direct console command lines that name the containing package folder—which need not be on the module search path in this mode:

```
$ python3 dir1 # Runs main.py (not init.py) Executing dir1.main.py
```

```
$ python3 dir1/dir2 Executing dir1.dir2.main.py
```

```
$ python3 dir1/dir2/mod.py # Runs nested mod.py Loading dir1.dir2.mod
```

Package `main.py` files are also run for the Python `-m` mode, which, as we've seen, locates an item on the module search path and runs it as a top-level script. Unlike direct console command lines, this mode kicks off the full package-import machinery and runs any `init.py` files along the path, but requires the package to be on the search path:

```
$ python3 -m dir1 # Runs both init.py and main.py Running dir1.init.py Executing dir1.main.py
```

```
$ python3 -m dir1.dir2 # And package must be on search path Running dir1.init.py Running dir1.dir2.init.py Executing dir1.dir2.main.py
```

```
$ python3 -m dir1.dir2.mod # Runs mod.py, and parents' init.py Running dir1.init.py Running dir1.dir2.init.py Loading dir1.dir2.mod
```

This is similar in spirit to app "bundles" on macOS and smartphones, which are really folders but can be run as an executable item. The roles for `main.py` files are many, but they're often used to provide a command-line interface to a package of tools. This way, you can import the package to use its tools in another pro

gram, but also launch it as a whole to use the tools in a standalone program. This file might also be used to host test code for the package.

A `main.py` file is also run if located in a ZIP file on the search path. As noted earlier, ZIP files are treated like a normal folder if encountered during a module search, but see Python's docs for more on this option.

One subtlety here: `intrapackage` (same-package) imports in `main.py` may need to use the package-relative `from` syntax we'll study ahead when run by the `-m` argument only. This syntax fails for launches from direct command lines, so you may need to choose a launch mode to support when a `main.py` requires same-package imports. As you'll find, code that wants to support both package and program usage may solve this dilemma with full-path imports.

中文翻译

最后，如果你希望包的使用者能像运行程序或脚本一样运行你的包，就在你想支持这种模式的每个文件夹中添加 `main.py` 文件；每次它们的容器文件夹被当作程序启动时，Python 都会自动运行这些文件。下面是这最后一部分如何扩展我们的包结构：

```
dir0/ dir1/ main.py # 每当 dir1 这个整体被运行时执行
init.py mod.py dir2/ main.py # 每当 dir1.dir2 这个整体被运行时执行
init.py mod.py 新增的 main.py 文件列在示例 24-5 和 24-6 中；它们很简单，好让我们专注于包本身。示例 24-5. dir1/main.py python print('Executing dir1.main.py') 示例 24-6. dir1/dir2/main.py
```

python print('Executing dir1.dir2.main.py') 存在 main.py 时，它会在多种启动方式下被自动运行，包括直接在控制台命令行中指明包文件夹的启动方式——在这种模式下，包无需位于模块搜索路径上：\$ python3 dir1 # 运行 main.py (而非 init.py) Executing dir1.main.py \$ python3 dir1/dir2 Executing dir1.dir2.main.py \$ python3 dir1/dir2/mod.py # 运行嵌套的 mod.py Loading dir1.dir2.mod 包的 main.py 文件也会在 Python 的 -m 模式下运行。正如我们所见，-m 模式在模块搜索路径上定位一个条目，并将其作为顶层脚本运行。与直接的控制台命令行不同，该模式会启动完整的包导入机制，并运行路径沿途的 init.py 文件，但要求包在搜索路径上：\$ python3 -m dir1 # 同时运行 init.py 和 main.py Running dir1.init.py Executing dir1.main.py \$ python3 -m dir1.dir2 # 且包必须在搜索路径上 Running dir1.init.py Running dir1.dir2.init.py Executing dir1.dir2.main.py \$ python3 -m dir1.dir2.mod # 运行 mod.py，并运行父级 init.py Running dir1.init.py Running dir1.dir2.init.py Loading dir1.dir2.mod 这在精神上与 macOS 和智能手机上的应用“包” (app bundles) 相似——它们本质上就是文件夹，却可以像可执行条目一样运行。main.py 文件的用途很多，但最常见的用途是为一个工具包提供命令行界面。这样一来，你既可以在另一个程序中 import 这个包来使用它的工具，也可以把整个包当作独立程序启动来使用这些工具。这个文件也可以用来存放包的测试代码。如果 main.py 位于搜索路径上的一个 ZIP 文件中，它同样会被运行。如前所述，模块搜索遇到 ZIP 文件时会把它们当作普通文件夹处理，但关于这个选项的更多内容请参阅 Python 文档。这里有一个微妙之处：main.

py 中的包内（同包）导入，在仅以 -m 参数运行时，可能需要使用我们后面将要学习的包相对 from . 语法。这种语法在直接命令行启动时会失败，所以当有一个 main.py 需要同包导入时，你可能需要选择要支持哪一种启动模式。正如你将会发现的，想要同时支持“包”与“程序”两种用法的代码，可以用完整路径导入来解决这个两难问题。

深度理解

- 核心概念：main.py = “包的入口脚本”。python dir1 与 python -m dir1 是两种不同的启动协议：前者只是“运行一个文件夹里的 main.py”（不触发包机制），后者先走完整导入流程（先 init.py 再 main.py）。

- 底层实现：直接命令行方式下，解释器把 main.py 当作顶层脚本运行，name == 'main'，它所在目录被作为“home”加入 sys.path；-m 方式下，解释器先以包导入的方式加载路径沿途所有 init.py，再以 name == 'main' 执行目标文件。两者对 sys.path 的构造也不同——所以同样代码在两种模式下行为可能不同。

- 设计原因：Python 想让你“一个包既能 import 又能运行”。类比 macOS/手机 App：一个文件夹，双击就运行，但内部文件也能被其他程序引用。CLI 工具（如 pip、http.server 脚本）常用 main.py 承载命令行入口。

- 实际问题：python -m 包是运行已安装包的标准方式（如 python -m pip install ...），因为 -m 会正确找到 site-packages 中的包并执行其 main.py。

- 初学者误区：把 main.py 与 init.py 混淆（一个管运行、

一个管导入初始化)；以为 `python dir1` 和 `python -m dir1` 等价——后者必须包在 `sys.path` 上且会先跑 `init.py`。

代码分析

```
# dir1/__main__.py — 24-5
print('Executing dir1.__main__.py')

# dir1/dir2/__main__.py — 24-6
print('Executing dir1.dir2.__main__.py')
```

解释器如何处理：`python dir1` 时，解释器把 `dir1/main.py` 当作顶层入口：编译、以 `name == 'main'` 执行，把 `dir1` 目录设为 `home`（加入 `sys.path`），完全不加载包机制——所以看不到 `Running dir1.init.py`。而 `python -m dir1` 时，解释器先按导入机制解析 `dir1`：加载 `init.py`（打印 `Running dir1.init.py`），再定位 `dir1/main.py` 并同样以 `name == 'main'` 执行。直接运行 `python dir1/dir2/mod.py` 则完全绕过包，`mod.py` 打印 `Loading dir1.dir2.mod`（首次导入效果），因为其顶层代码被当脚本执行。

24.11 为什么要用包？ (Why Packages?)

英文原文

Now that you've seen how to use and create packages, you might be wondering why in the world anyone would go to all the bother. It's a valid question. In tru

th, and despite what you may have heard, packages are completely optional: they are probably overkill early in your coding journey, and even later, you can write amazing programs without them. In general, though, packages are useful to know about because they help make names unique in both larger programs and libraries published for others to use. By organizing your code as a package folder, you can be fairly sure that the names of its files won't clash with those of other software on hosting devices. This is the same namespace-segregation role that file-based modules and local scopes in functions play, but extended to the filesystem: because references to your package are qualified by the name of the package's folder, there's less risk of same-name collisions. In addition, packages can make imports more descriptive, ease the task of tracking down the locations of variables in code files, and simplify your module search-path settings. The only time package imports are actually required, however, is to resolve ambiguities that may arise when multiple programs with same-named files are installed on the same machine. This is something of an install issue, but it can also become a concern in general practice—especially given the tendency of developers to use simple and similar names for module files. To help you better understand why all these benefits matter, let's take a brief detour into the land of the abstract.

中文翻译

看完了如何使用和创建包，你可能会想：到底有谁会费这么大的劲呢？这是个合理的问题。事实上，不管你听说了什么，包完全是可选的：在编程之路的早期它很可能是杀鸡用牛刀，即使到了后期，不用包你也能写出很棒的程序。但总体而言，了解包是有用的，因为它有助于让名字在更大的程序、以及发布给别人使用的库中保持唯一。把代码组织成包文件夹，你可以比较有把握地说：它的文件名不会与宿主设备上其他软件的文件名冲突。这与基于文件的模块、函数中的局部作用域所扮演的"命名空间隔离"角色如出一辙，只是扩展到了文件系统层面：因为对你的包的引用都带有包文件夹名的限定前缀，同名冲突的风险就小得多。此外，包还能让导入更具描述性、方便追踪代码文件中变量的位置、简化模块搜索路径的设置。不过，包导入真正必需的唯一场景，是解决同一台机器上安装了多个同名文件程序时可能出现的歧义。这多少算个安装问题，但在日常实践中也可能成为隐患——尤其是考虑到开发者给模块文件起名时，偏爱简单而相似的名字。为了让你更好地理解这些好处为什么重要，让我们暂时进入抽象的领域走一趟。

深度理解

·核心概念：包存在的价值 = 命名空间隔离。模块隔离作用域，函数局部变量隔离名字，包则把这种隔离延伸到文件系统——目录名成为全局唯一的"姓氏"。

·底层实现：导入的全局名字空间是共享的：`import utilities` 之后，`utilities` 这个名字指向哪个模块，完全取决于 `sys.path` 中哪个目录先被找到。包让引用变成 `system1.ut`

ilities 这种限定名，绕开了全局名字的竞争。

·设计原因：早期 Python 没有包时，"两个程序都有 `utilities.py`" 就只能靠改 `sys.path` 或改文件名来规避——都是脆弱的方案。包提供了结构性解决方案：把"安装在哪"与"叫什么"解耦。

·实际问题：`sys.path` 是线性扫描的（从左到右），两个同名模块必然只有排在前面的那个被找到；在脚本里临时改 `sys.path` 既麻烦又易错，改 `PYTHONPATH` 又无法在同一文件里同时用两个版本。包一次解决。

·初学者误区：以为包是"必学必用"的——Lutz 明确说它是可选的，早期学习阶段可以完全跳过；但认识它，是为了理解你日后必用的标准库/第三方库的组织方式。

24.12 两个系统的故事 (A Tale of Two Systems)

英文原文

To illustrate the roles of packages, suppose that a programmer develops a Python program that contains a file called `utilities.py` for common utility code, and a top-level file named `main.py` that users launch to start the program. All over this program, its files say `import utilities` to load and use the common code.

When the program is installed, it unpacks all its files into a single directory named `system1` on the target

machine that looks like this:

```
system1/ utilities.py # Common utility functions, clas  
ses main.py # Launch this to start the program other.  
py # Import utilities to load my tools
```

Now, suppose that a second programmer develops a different program with files also called `utilities.py` and `main.py`, and again uses `import utilities` throughout the program to load its own common code file. When this second system is fetched and installed on the same computer as the first system, its files will unpack into a new directory called `system2` on the host—ensuring that they do not overwrite same-named files from the first system:

```
system2/ utilities.py # Common utilities main.py # La  
unch this to run other.py # Imports utilities
```

So far, there's no problem: both systems can coexist and run on the same computer. In fact, you won't even need to configure the module search path to use these programs on your computer—because Python always searches the home directory first (that is, the directory containing the top-level file), imports in either system's files will automatically see all the files in that system's own directory. For instance, if you run `system1/main.py`, all imports will search `system1` first. Similarly, if you launch `system2/main.py`, `system2` will be searched first instead. Remember, module search path settings are only needed to import across directo

ry boundaries.

However, suppose that after you've installed these two programs on your machine, you decide that you'd like to use some of the code in each of the `utilities.py` files in a system of your own. It's common utility code, after all, and Python code by nature "wants" to be reused. In this case, you'd like to be able to say the following from code that you're writing in a third directory to load one of the two files:

```
python import utilities utilities.func('hack')
```

Now the problem starts to materialize. To make this work at all, you'll have to set the module search path to include the directories containing the `utilities.py` files. But which directory do you put first in the path—`system1` or `system2`?

The issue here is the linear nature of the `sys.path` search path. It is always scanned from left to right, so no matter how long you ponder this dilemma, you will always get just one `utilities.py`—from the directory listed first (leftmost) on the search path. As is, you'll never be able to import this file from the other directory at all.

You could try changing `sys.path` within your script before each import operation, but that's both extra work and error-prone. And changing `PYTHONPATH` before each Python program run is too tedious, and won't allow you to use both versions in a single file in an ev

ent. By default, you're stuck.

This is the issue that packages actually fix. Rather than installing programs in independent directories listed on the module search path directly and individually, you can package and install them as subdirectories under a common root. For instance, you might organize all the code in this example as an install hierarchy that looks like this:

```
root/ system1/ utilities.py main.py other.py system2/  
utilities.py main.py other.py mycode/ # Here or elsew  
here myfile.py # Your new code here
```

Now, add just the common root directory to your search path. If your code's imports are all relative to this common root, you can import either system's utility file with a package import—the enclosing directory name makes the path (and hence, the module reference) unique. In fact, you can import both utility files in the same module, if you use an import statement and repeat the full path each time you reference the utility modules:

```
python import system1.utilities # Import from one package  
import system2.utilities # Import from another  
system1.utilities.function('hack') # And use names from  
either system2.utilities.function('code') # Or both!
```

In short, the names of the enclosing directories here make the module references unique.

Note that you have to use `import` instead of `from` with packages only if you need to access the same attribute name in two or more paths. If the name of the called function here were different in each path, you could use `from` statements to avoid repeating the full package path whenever you call one of the functions, as described earlier; as also mentioned, the `as` extension in `import` and `from` can be used to provide unique synonyms too.

Technically, in this case, the `mycode` folder doesn't have to be under `root`—just the packages of code from which you will import. Because you never know when your own modules might be useful in other programs, though, placing them under the common `root` directory, too, may avoid similar name-collision problems in the future.

Importantly, path configuration also becomes simple if you're careful to unpack all your Python systems under a common root like this; you'll only need to add the common root directory once. Moreover, both of the two original systems' imports will keep working unchanged. Because their home directories are searched first, the addition of the common root on the search path is irrelevant to code in `system1` and `system2`; they can keep saying just `import utilities` and expect to find their own files when run as programs. As you'll see ahead, though, if they are also used as packages, they may need to use `from . package-relative imports`

(and main.py could be main.py).

Finally, keep in mind that even if you never create packages of your own, you'll probably use standard-library and third-party tools that do. As a random sample of standard-library packages:

```
python from email.message import Message # Email
parsing/construction from tkinter.filedialog import as
kopenfilename # Portable GUI toolkit from http.serve
r import CGIHTTPRequestHandler # Web-server utiliti
es
```

By bundling their code in a package, such tools become more self-contained, and avoid name conflicts. Whether you opt to do the same for your own code is up to you, but the next few sections dig deeper for if and when you opt-in.

中文翻译

为了说明包的作用，假设一位程序员开发了一个 Python 程序：其中有一个 `utilities.py` 文件存放通用工具代码，还有一个顶层文件 `main.py` 供用户启动程序。整个程序的文件都写着 `import utilities` 来加载和使用这段公共代码。程序安装时，所有文件被解压到目标机器上一个名为 `system1` 的目录中，结构如下：

```
system1/ utilities.py # 通用工具
函数、类main.py # 启动程序时运行它other.py # 用 imp
ort utilities 来加载我的工具
```

现在假设第二位程序员开发了另一个程序，文件同样叫 `utilities.py` 和 `main.py`，程序内也处处 `import utilities` 来加载自己的公共代码文件。当第二个系统被下载并安装到与第一个系统相同的电脑上时，它的文件会解压到宿主上一个新目录 `system2`——这保证了

它不会覆盖第一个系统的同名文件：`system2/ utilities.py`
通用工具`main.py` # 运行它来启动`other.py` # 导入 `utilities` 到目前为止没有问题：两个系统可以在同一台电脑上共存并运行。事实上，你甚至不需要配置模块搜索路径就能使用这两个程序——因为 Python 总是先搜索 `home` 目录（即包含顶层文件的目录），任一系统文件中的导入都会自动看到该系统自己目录中的所有文件。例如，运行 `system1/main.py`，所有导入都会先搜 `system1`；同样，启动 `system2/main.py`，就会先搜 `system2`。记住，只有跨目录导入才需要配置模块搜索路径。然而，假设把这两个程序装到机器上之后，你想在自己写的一个系统里使用两个 `utilities.py` 文件中的部分代码。毕竟这是通用的工具代码，而 Python 代码天生“渴望”被复用。这种情况下，你希望能在自己写到第三个目录里的代码中，通过下面这种方式加载两个文件之一：`python import utilities utilities.func('hack')` 现在问题开始显现。要让这个导入成功，你必须把包含 `utilities.py` 的目录加入模块搜索路径。但哪个目录该放在路径的最前面——`system1` 还是 `system2`？问题出在 `sys.path` 搜索路径的线性特性上。它总是从左到右扫描，所以无论你纠结多久，永远只能得到一个 `utilities.py`——来自搜索路径上最前面（最左）列出的目录。照此下去，你永远无法从另一个目录导入这个文件。你也可以尝试在脚本里每次导入前修改 `sys.path`，但那既费事又容易出错。而每次运行 Python 程序前改 `PYTHONPATH` 又太繁琐，而且也没办法在同一个文件里同时使用两个版本。默认情况下，你无路可走。这正是包真正解决的问题。与其把程序直接、逐个地安装到模块搜索路径上彼此独立的目录里，不如把它们打包安装到一个公共根目录下的子目录中。例如，你可以把这个例子里

的所有代码组织成如下安装层级： root/ system1/ utilities.py main.py other.py system2/ utilities.py main.py other.py mycode/ # 放这里或其他地方 myfile.py # 你的新代码在这里现在，只需把公共的 root 目录加入搜索路径。如果你的代码中的导入都相对于这个公共根目录，那么你可以用包导入来加载任一系统的工具文件——外层目录名让路径（进而让模块引用）变得唯一。事实上，如果你用 import 语句、并在每次引用工具模块时重复完整路径，你甚至可以在同一个模块中同时导入两个工具文件： python import system1.utilities # 从一个包导入 import system2.utilities # 从另一个包导入 system1.utilities.function('hack') # 使用任一包中的名字 system2.utilities.function('code') # 或者两个都用！简而言之，这里外层目录的名字让模块引用变得唯一。注意：只有当你需要在两个或多个路径中访问同一个属性名时，才必须对包使用 import 而不是 from 。如果这里被调用的函数名在各个路径中各不相同，你就可以用 from 语句，避免每次调用某个函数时重复完整的包路径（如前所述）；同样如前所述，import 和 from 中的 as 扩展也可以用来提供唯一的别名。严格来说，在这个例子里， mycode 文件夹不必放在 root 之下——只需把你将要导入的代码包放在那里即可。不过，因为你永远不知道自己的模块将来会不会在别的程序中有用，把自家模块也放到公共根目录下，也许能避免将来类似的同名冲突问题。重要的是，如果你像这样把所有 Python 系统都解压到公共根目录下，路径配置也会变得很简单——你只需把公共根目录添加一次。而且，最初两个系统里的导入都会原样继续工作。因为它们的 home 目录会被优先搜索，公共根目录加进搜索路径对 system1 和 system2 里的代码毫无影响；作为程

序运行时，它们可以继续只写 `import utilities`，并期待找到自己的文件。不过正如你稍后会看到的，如果它们也被当作包来使用，就可能需要使用 `from .` 包相对导入（并且 `main.py` 可以改成 `main.py`）。最后，请记住：即使你从不创建自己的包，你也很可能使用采用了包机制的标准库和第三方工具。标准库包的一个随机样本：`python from email.message import Message # 电子邮件解析/构造`
`from tkinter import filedialog import askopenfilename # 可移植 GUI 工具包`
`from http.server import CGIHTTPRequestHandler # Web 服务器工具`通过把代码打包，这类工具更加自包含，也避免了名字冲突。是否为自己的代码也这样做，由你自己决定；但接下来几节会深入探讨，供你选择加入时参考。

深度理解

- 核心概念：`sys.path` 的线性扫描决定了"同名即冲突"；包通过外层目录名把同名模块变成不同引用。这是包唯一"必需"的场景。
- 底层实现：导入 `system1.utilities` 时，解释器在 `root` (`sys.path` 条目) 下找到 `system1` 目录，在其内找到 `utilities.py`，完整模块名是 `system1.utilities`，与 `system2.utilities` 是两个互不干扰的缓存键 (`sys.modules` 键名不同)。这正是"限定名 = 全局唯一键"的机制。
- 设计原因：把"安装目录"与"命名空间"绑定：只要所有程序共享一个公共根，路径配置只需一次；两个老系统的内部 `import utilities` 也完全不受影响 (`home` 目录优先)。这是一种向后兼容的结构性修复。

·实际问题：真实世界的例子就是 pip 安装的包全在 site-packages 下；email.message、tkinter.filedialog 等标准库包同样如此。若没有包，两个库都叫 message.py 就会互相覆盖。

·初学者误区：以为必须把所有代码都放进包；以为 from 在包场景中"一定更优"——当两个路径有同名的函数/变量需要同时使用时，from 会把后导入的覆盖先导入的，必须用 import + 完整路径（或 as 别名）。

代码分析

```
import system1.utilities          # system1/utilities.py...
import system2.utilities          # system2/utilities.py
system1.utilities.function('hack') #
system2.utilities.function('code') #

# from
from system1.utilities import f1
from system2.utilities import f2
f1('hack'); f2('code')

# functionfrom import as
from system1.utilities import function #
from system2.utilities import function # function system2 ...
```

解释器如何处理：每次 import 都会为模块生成唯一的完整名称（system1.utilities），存入 sys.modules 字典；属性访问 system1.utilities.function 沿着模块对象的属性链查找，绝无歧义。而 from ... import function 是往当前命名空间写一个裸名字 function，第二次导入会把第一次的覆盖掉——这就是"裸名冲突"的重演。as 别名（from system2.utilities import function as f2）则是给裸

名重新起名以绕开冲突的官方解法。

24.13 init.py 文件的作用 (The Roles of init.py Files)

英文原文

Now that we've seen the basics and addressed the why of packages, let's explore some of the details behind their usage. The `init.py` files in our opening demo were simple, but these files can contain arbitrary Python code, just like normal module files. Their names are special because their code is run automatically the first time a Python program imports a directory, and thus serves primarily as a hook for performing initialization steps required by the package. These files can also be completely empty, though, and sometimes have additional roles. Specifically, the `init.py` file serves as a hook for package initialization-time actions, generates a module namespace for a directory, declares a directory as a "normal" Python package, and implements the behavior of `from` statements when used for folders in package imports: Package initialization — The first time a Python program imports through a directory, it automatically runs all the code in the directory's `init.py` file. Because of that, these files are a natural place to put code to initialize the state required by files in a package. For instance, a package might u

se its initialization file to create required data files, open connections to databases, and so on. Typically, `init.py` files are not meant to be useful if executed directly (that's what `main.py` is for); instead, they are run automatically when a package is first imported for use elsewhere.

Module namespace initialization — As noted earlier, in the package import model, the directory paths in your script become real nested object paths after an import. For instance, after an import in the preceding demo, the expression `dir1.dir2` works and returns a module object whose namespace contains all the names assigned by `dir2's init.py` initialization file. The variable assignments in such files provide attribute namespaces for module objects created for folders, which have no real associated module file, and would otherwise have no place to define names.

Package indicator for search — Package `init.py` files are also partly present to declare that a directory is a Python package. In this role, these files serve to prevent directories with common names from unintentionally hiding true modules that appear later on the module search path. Without this safeguard, Python might pick a directory that has nothing to do with your code, just because it appears nested in an earlier directory on the search path. As you'll see later, the generalizations added for namespace packages subsume some of this role algorithmically, by scanning ahead on the search path to find later items before settling on sim

ple folders. Package `init.py` files, though, still give a package higher priority than a same-name folder elsewhere on the search path. from statement behavior

— As an advanced feature, a list of name strings named `__all__` at the top level of an `init.py` file defines what is exported for the `from` statement form. Specifically, the `__all__` list in an `init.py` file is taken to be the names of nested submodules that should be automatically imported when `from` is used on the package directory itself. If `__all__` is not set, the `from` does not automatically load submodules nested in the package directory; instead, it loads just names defined by assignments in the directory's `init.py` file, including any submodules explicitly imported by code in this file. For instance, `from submodule import X` in a directory's `init.py` makes the name `X` available in that directory's namespace without an `all`. You'll see an example of the `__all__` list later, and more coverage of it in Chapter 25 where you'll find that it also serves to declare `from` exports of simple file-based modules, not just packages, and is part of the larger topic of data hiding. You can also simply leave `init.py` files empty to give your package precedence over namespace packages during an import search, but to understand why that matters, we need to move ahead.

NOTE — Classes conflation caution: As a preview, package `init.py` files are not the same as the class `__init__` constructor methods you'll meet in the next part of this book. The former are files of code run wh

en imports first step through a package folder in a program run, while the latter are functions called whenever an instance is created. Both have initialization roles and are optional, but they are otherwise very different—despite their names.

中文翻译

现在我们已经了解了基础知识、回答了“为什么用包”的问题，接下来探讨一下它们用法背后的一些细节。开场演示中的 `init.py` 文件很简单，但这些文件其实可以包含任意 Python 代码，就像普通模块文件一样。它们的名字特殊，是因为其代码会在 Python 程序首次导入某个目录时被自动运行，因此主要充当执行包所需初始化步骤的钩子。当然，这些文件也可以完全为空，有时还有其他作用。具体来说，`init.py` 文件充当包初始化时动作的钩子、为目录生成模块命名空间、把目录声明为“常规”Python 包，并实现包导入中文件夹的 `from` 语句行为：包初始化（Package initialization）——Python 程序首次“经由”某个目录导入时，会自动运行该目录 `init.py` 文件中的全部代码。因此，这些文件是把包内文件所需的状态初始化代码放进去的自然之选。例如，包可以用它的初始化文件创建所需的数据文件、打开数据库连接等。通常，`init.py` 文件并不是为了被直接执行而设计（那是 `main.py` 的职责）；相反，它们是在包首次被导入以供他处使用时自动运行的。模块命名空间初始化（Module namespace initialization）——如前所述，在包导入模型中，脚本里的目录路径在导入后会变成真实的嵌套对象路径。例如，前面演示中一次导入之后，表达式 `dir1.dir2` 就能工作并返回一个模块对象，其命名空间包含 `dir2` 的 `init.py`

初始化文件赋值的所有名字。这类文件中的变量赋值，为那些"没有真实关联模块文件"的文件夹模块对象提供了属性命名空间——否则它们没有任何地方可以定义名字。搜索时的包指示器（Package indicator for search）——包的 `__init__.py` 文件在某种程度上还用于声明"这个目录是一个 Python 包"。在这一角色中，这些文件可以防止名字常见的目录无意中遮蔽模块搜索路径上更靠后的真正模块。没有这道防线，Python 就可能因为某个目录出现在搜索路径更靠前目录的嵌套中，就选中一个与你的代码毫无关系的目录。正如你稍后将看到的，为命名空间包所做的泛化扩展在算法上接管了部分职责——在锁定普通文件夹之前，会先沿搜索路径向前扫描以找到更靠后的条目。不过，`__init__.py` 文件仍能让一个包比搜索路径上其他位置的同名文件夹获得更高的优先级。`from` 语句行为（`from statement behavior`）——作为一个高级特性，在 `__init__.py` 文件顶层定义的名字字符串列表 `__all__` 定义了 `from` 语句形式会导出什么。具体来说，`__init__.py` 中的 `__all__` 列表被当作"对包目录本身使用 `from` 时，应当自动导入的嵌套子模块名字"。如果未设置 `__all__`，`from` 不会自动加载包目录中嵌套的子模块；它只加载该目录 `__init__.py` 文件中由赋值定义的、以及由该文件代码显式导入的名字。例如，在目录的 `__init__.py` 中写 `from submodule import X`，就能在没有 `__all__` 的情况下让名字 `X` 在该目录的命名空间中可用。稍后你会看到 `__all__` 列表的示例，第 25 章还会有更完整的讲解——届时你会发现它同样用于声明普通基于文件的模块（而不只是包）的 `from` 导出，并且属于更大的"数据隐藏"（`data hiding`）主题的一部分。你也可以干脆让 `__init__.py` 文件保持为空，让包在导入搜索中优先于命名空间包；但要理解为什么这很重要，我们需要继续往下读。注意

——类混淆警示 (Classes conflation caution)：提前预告一下，包的 `init.py` 文件与本书下一部分会遇到的类 `init` 构造方法不是一回事。前者是“程序运行时导入首次踏足某个包文件夹时执行”的代码文件；后者是“每次创建实例时调用”的函数。两者都承担初始化职责、也都是可选的，但除此之外它们非常不同——尽管名字相似。

深度理解

·核心概念：`init.py` 有四个角色：初始化钩子、文件夹的命名空间载体、包的“身份声明”、`from` 的导出控制器。其中“身份声明”是 Python 3 之前包机制的核心。

·底层实现：导入系统在路径探查时，对每个候选目录检查目录名/`init.py` 是否存在：存在 → 按“常规包”处理（优先）；不存在且是目录 → 记为命名空间包候选（优先级较低）。`all` 则是在 `from` 执行时被读取的约定属性。

·设计原因：早期 Python (3.3 之前) 把 `init.py` 当作“这个目录值得被当包处理”的唯一标志，防止同名的普通数据目录被误当作包。如今它退化为优先级标志，但“声明式初始化”的价值依然存在。

·实际问题：包的初始化状态（配置文件、连接、注册表）、`from` 包 `import` 的受控导出——没有 `init.py` 都无法优雅实现。

·初学者误区：把 `init.py` 和类 `init` 混淆（一个是文件、一个是方法，名字撞车纯属历史巧合）；以为 `init.py` 里 `import` 的子模块会自动出现在包命名空间——只有显式 `import` (或 `all`) 才会。

代码分析

```
# __init__.py
from submodule import X      # X from import *
import submodule2             # submodule2

# __all__ from import *
__all__ = ['mod', 'var']      # /
var = 'Python'
```

解释器如何处理：当代码写 `from 包 import` 时，解释器检查包的模块对象是否定义了 `all`：有——按列表导入名字（列表里若是子模块名，会触发导入并绑定）；没有——只导入 `init.py` 命名空间中不以单下划线开头的名字（包括显式 `from submodule import X` 带来的 `X`，以及 `var` 这类赋值）。注意 `all` 只是导入机制的约定，不会阻止 `import 包.mod` 这样的显式访问——它只管 `from` 的默认行为，这也是“数据隐藏”（约定大于强制）思想的体现。

24.14 包相对导入 (Package-Relative Imports)

英文原文

The coverage of package imports so far has focused on importing package files from outside the package. Within the package itself, imports of same-package files can use the same full path syntax as imports from outside the package—and as you'll see, sometimes s

hould. However, files within a package can also make use of intrapackage syntax that makes imports relative to the package itself, and can ensure that imports in a package load the package's own files.

中文翻译

到目前为止，我们对包导入的讲解聚焦于从包外部导入包文件。在包内部，对同包文件的导入可以使用与包外导入相同的完整路径语法——正如你将要看到的，有时应该这么做。不过，包内的文件也可以使用一种包内语法，让导入相对于包自身进行，从而确保包中的导入加载的是包自己的文件。

深度理解

- 核心概念：导入有"外部视角"（绝对路径导入）与"内部视角"（相对导入）两种；本书从"为什么需要内部视角"讲起——确保包加载自己的模块。
- 底层实现：两种语法最终都走同一套导入机制，区别只在于查找起点：绝对导入从 `sys.path` 开始，相对导入从当前包的 `path` 开始。
- 设计原因：包发布给别人后，你无法控制使用者的搜索路径配置；相对导入让包"自带安全感"，不受宿主环境影响。
- 实际问题：两个包都带 `utils.py` 时，绝对导入 `import utils` 可能命中别人的模块；相对导入 `from . import utils` 永远命中自己。

·初学者误区：以为相对导入是"缩略写法"——它不只是短，而是查找范围不同，这是两套语义。

24.15 相对导入与绝对导入 (Relative and Absolute Imports)

英文原文

The code behind this is straightforward. Imports run by files used as part of a package can use special syntax like the following, which works only in files used as package components, and only in from statements, not import: `python from . import module` # Import a module in this package (only) `from .module import name` # Import a name from a module in this package `from .. import module` # Import a module sibling of the parent folder `from ..module import name` # Import a name from a parent-sibling module This syntax wouldn't make sense in `import`, because that statement assigns modules to simple names, not paths. In a `from`, though, when the source of the import begins with (or is only) dots like this, the import is known as relative—it identifies an item relative to the folder of the enclosing package itself. This name comes from the similarity to relative filename paths, which reference a file per the current working directory (CWD), as covered in Chapter 9. Much as in filenames, `"."` means the immediately enclosing package folder, and `".."` mea

ns its parent folder another level up (and each additional dot means one level higher, though this is rarely used). Here, though, the effect names an item relative to an importer's package, not a content file relative to the CWD. Importantly, relative imports within a package search only the referenced package: they never check the folders listed in `sys.path` as normal imports do (and skip the built-in and frozen modules precheck). Moreover, relative-import syntax can be used only when a file is being utilized as part of a package, and fails otherwise. This has consequences both good and bad that we'll explore in a moment. Imports in files being used as part of a package can still be coded without dots as usual, in both `import` and `from`:

```
python import module # Import a module from a folder in sys.path
from module import name # Import a name from the same
```

Sans leading periods like this, an `import` is known as absolute—it identifies an item that's located in a folder listed in `sys.path`. This is the normal behavior of imports in Python, and there's really nothing "absolute" about it per the filename analogy (absolute filename paths give a complete filesystem path). In fact, these "absolute" imports are really relative to an entry in the module search path, but we're stuck with the terminology today. Also importantly, absolute imports within a package do not automatically check the package itself: they skip the package and move right to a search of folders on `sys.path`. As we've

e seen, `sys.path` may include the package's folder any how, by virtue of the REPL's CWD or the home folder of the top-level script, and `PYTHONPATH` or other settings. By themselves, though, absolute imports don't check the package for imports run in package files.

中文翻译

背后的代码很直接。作为包的一部分被使用的文件，其执行的导入可以使用下面这样的特殊语法——它只对作为包组件的文件有效，而且只适用于 `from` 语句，不适用于 `import`：
`python from . import module` # 导入本包中的模块
(仅限本包) `from .module import name` # 从本包中的模块导入一个名字
`from .. import module` # 导入父文件夹的兄弟模块
`from ..module import name` # 从父级兄弟模块导入一个名字
这种语法在 `import` 中没有意义，因为 `import` 语句是把模块赋给简单名字，而不是路径。但在 `from` 中，当导入源以这样的点号开头（或全由点号构成）时，这个导入被称为相对导入——它相对于所在包自身的文件夹来标识条目。这个名字来源于与相对文件名路径的相似性——相对文件名路径是相对当前工作目录（CWD）引用文件的，第 9 章讲过。和文件名一样，`"."` 表示紧邻的包文件夹，`".."` 表示再上一层的父文件夹（每多加一个点就再高一层，不过这很少用）。不过在这里，其效果是相对导入者的包来命名条目，而不是相对 CWD 命名内容文件。重要的是，包内的相对导入只搜索被引用的包：它们从不检查 `sys.path` 中列出的文件夹（普通导入会检查，还跳过内置和冻结模块的预检查）。而且，相对导入语法只有在文件被当作包的一部分使用时才可用，否则就会失败。这带来好与坏两方面

的后果，我们马上探讨。作为包的一部分被使用的文件，其导入仍然可以照常不带点号书写，`import` 和 `from` 都可以：

- `python import module` # 从 `sys.path` 中的某个文件夹导入模块
- `from module import name` # 从同一处导入名字

像这样不带前导点号的导入被称为绝对导入——它标识的条目位于 `sys.path` 列出的某个文件夹中。这是 Python 导入的正常行为；按文件名的类比来说，它其实没什么“绝对”可言（绝对文件名路径给的是完整文件系统路径）。事实上，这些“绝对”导入真正是相对模块搜索路径上的某个条目而言的，但今天的术语已经这样定下来，我们只能接受。同样重要的是，包内的绝对导入不会自动检查包自身：它们跳过包，直接转向 `sys.path` 上的文件夹搜索。正如我们所见，由于 REPL 的 CWD、顶层脚本的 `home` 文件夹以及 `PYTHONPATH` 等设置，`sys.path` 本来就可能包含包的文件夹。但就其本身而言，在包文件内执行的绝对导入并不会检查包。

深度理解

- 核心概念：点号语法 = 相对导入；无点号语法 = 绝对导入。点的个数表示“向上”的层数：`.` 是本层，`..` 是上一层。
- 底层实现：相对导入在字节码层最终仍会转化为一个完整模块名（解释器把“当前包名 + 点号层级”拼接出来再走常规导入流程）。Python 3 中相对导入要求文件必须有 `package` 信息（由包机制注入）——顶层脚本（`name == 'main'` 且无包上下文）没有这个信息，所以 `from . import x` 直接报 `ImportError: attempted relative import with no known parent package`。

·设计原因：相对导入的"仅查本包"语义是故意的：避免包被宿主机上无关的同名模块"投毒"（搜索路径你管不着），让包自包含、可预测。

·实际问题：包内的绝对导入可能"看不见自己"——`import utils` 在包文件里不会自动去包内找 `utils`（除非包的容器恰好在 `sys.path` 上）。所以包内互相引用要么用相对导入，要么用从包根开始的完整路径导入。

·初学者误区：以为 `import .module` 合法——点号语法只允许用于 `from`；以为相对导入可以在脚本顶层文件里用——顶层脚本不是包的一部分，`from . import x` 会立刻失败。

代码分析

```
from . import module          # " module "
from .module import name      # module name
from .. import module         #
from ..module import name     #

import module                  # sys.path module
from module import name       # from
```

解释器如何处理：编译器遇到 `from .module import name` 时，把点号层级与当前模块的 `package`（例如 `dir1.dir2`）拼接，解析出完整模块名 `dir1.dir2.module`，然后走常规查找流程——但查找候选集是当前包的 `path`（对命名空间包则是多个目录），而非 `sys.path`。若当前文件没有包上下文（`package` 为空或未设置，典型如直接运行脚本），解析立即失败并抛出 `ImportError: attempted relative import with no known parent package`。而 `i`

import module 这类无点号语句的候选集永远是 `sys.path`——它甚至不会先看自己所在的包目录。

24.16 相对导入的理由与权衡 (Relative-Import Rationales and Trade-Offs)

英文原文

So why the dots? In short, relative imports allow a package to ensure that its imports will load its own modules. Because relative imports search only the package itself, they won't inadvertently load a same-named but unrelated module elsewhere on the host that happens to be accessible through `sys.path`. This in turn makes the package more self-contained: without relative imports, such an unrelated module might break the package's code; with them, packages are less reliant on client search-path settings that they cannot predict or control. The downside is that this model is an all-or-nothing proposition. You essentially must choose a mode for your files—package or program. Relative imports give visibility to the package itself, but cannot be used in nonpackage mode, and absolute imports can be used in nonpackage mode, but do not give visibility to the package itself. This combination seems a catch-22 that limits your code's utility. That being said, if you're coding a library of tools, this may be a nonissue: you can adopt relative imports through

out your package, and provide a `main.py` file to run the package as a program with the `-m` switch. If you're writing a more traditional folder of code that you want to use arbitrarily, though, your options appear to be limited. One easy solution to this dilemma is to simply avoid relative imports altogether, and use full, explicit, and "absolute" package-import paths everywhere in your code. That is, use imports that list the path to the desired item from and including the package's root folder itself:

```
python import package.module # Import a module in this package from package
import module # Same as: from . import module
from package import module # Same as: from . import module
from package.module import name # Same as: from .module import name
```

For this to work, the package's root folder must reside in a folder listed on `sys.path`, but this will be the normal case—there's no way for clients to import the package's code otherwise. This also doesn't directly support exotic imports like `".."` parent siblings, but these seem likely to be rare (e.g., only one standard-library tool uses them today). With this nonrelative equivalence, code folders can be used more flexibly, and both direct command lines and the `-m` module-mode switch can be used to launch files in the package as programs. The next section shows how.

中文翻译

那么为什么要用点号？简而言之：相对导入让包能够确保自己的导入加载的是自己的模块。因为相对导入只搜索包本身

，就不会无意中加载宿主机上恰好能通过 `sys.path` 访问到的、同名却不相关的模块。这反过来让包更加自包含：没有相对导入，这种不相关的模块可能破坏包的代码；有了相对导入，包就不那么依赖客户方那些自己无法预测也无法控制的搜索路径设置。缺点是这个模型是全有或全无式的命题。你基本上必须为文件选择一种模式——是“包”还是“程序”。相对导入看得见包自身，但不能在非包模式下使用；绝对导入可以在非包模式下使用，却看不见包自身。这个组合看起来像个“第 22 条军规”，限制了代码的用途。话虽如此，如果你写的是一个工具库，这可能根本不是问题：你可以在整个包中使用相对导入，并提供一个 `main.py` 文件，用 `-m` 开关把包作为程序运行。但如果你写的是一个更传统的代码文件夹，想要随意地使用它，那你的选择看起来就很有限制了。解决这个两难问题的一个简单办法是：干脆完全避免相对导入，在代码中处处使用完整、显式、“绝对”的包导入路径。也就是说，使用那些“从包根文件夹开始（并包含它）列出到目标条目路径”的导入：`python import package.module # 导入本包中的模块`
`from package import module # 等价于：from . import module`
`from package.module import name # 等价于：from .module import name`要让这工作，包的根文件夹必须位于 `sys.path` 所列的某个文件夹中——但这本来就是常态：否则客户方根本没法导入包的代码。这种方式也不直接支持“..”父级兄弟这类奇特的导入，但这类需求似乎很少见（例如，今天只有极少数标准库工具在用）。有了这种“非相对”的等价写法，代码文件夹可以更灵活地使用，直接命令行和 `-m` 模块模式开关都可以把包中的文件当作程序启动。下一节将展示具体做法。

深度理解

·核心概念：相对导入是一把双刃剑——安全（只找自己）与受限（不能脱离包）。而"完整路径导入"（`import package.module`）是两种模式通吃的折中方案。

·底层实现：`from package import module` 与 `from . import module` 解析出的最终模块名相同（`package.module`），差异只在查找起点：前者走 `sys.path` 找到 `package` 根（根必须在路径上），后者直接从当前包的 `path` 出发。结果等价，但前者在非包模式下也能用。

·设计原因：Python 3 统一了导入语义（相对导入必须显式加点），消除了 Python 2 时代隐式相对导入带来的混乱；"全有或全无"是这套干净语义的代价。

·实际问题：想同时支持"作为程序运行"（`python dir1`、`python -m dir1`）与"作为包被导入"（`import dir1.mod`），最稳妥的写法就是完整路径导入——它不依赖运行模式。

·初学者误区：以为 `from . import module` 只是 `from package import module` 的简写——两者运行时行为有细微差异（前者的查找不经过 `sys.path`，更"防毒"），但通常可互换；以为相对导入能在任何脚本里用——顶层脚本（无包上下文）一概失败。

代码分析

```
# ""/
from . import module          #
from package import module    # package sys.pa...
import package.module         # package.module ...

#
# from . import module — <>.module
# from package import module — package.module packag...
```

解释器如何处理：编译 `from . import module` 时，编译器使用当前模块的 `package` 属性（例如 `dir1`）解析出全名 `dir1.module` 交给导入系统；导入系统在 `dir1.path` 中查找 `module`。编译 `from package import module` 时，导入系统先沿 `sys.path` 找到 `package` 目录（含 `init.py`），再在其 `path` 中找 `module`。两者最终在 `sys.modules` 中注册的键名一致（假设当前包就是 `package`），所以混用不会产生两个副本；但在“直接命令行启动”模式下，`from . import module` 因缺少包上下文直接报错，而 `from package import module` 依然可用——这就是下一节实验要验证的核心结论。

24.17 包相对导入实战 (Package-Relative Imports in Action)

英文原文

To demo all of the foregoing points, let's return to the simple package we coded at the start of this chapter. As you'll recall, its `main.py` file is run automatically when the folder is run as a bundle, but it will also pla

y the role of any top-level file in the folder launched as a script. When we last met this file, it simply contained a print; here's a refresher of the last-known state of the files we'll be using here so you don't have to flip back: `python # prior dir1/mod.py var = 'hack'print('Loading dir1.mod') # prior dir1/main.py print('Executing dir1.main.py')`

中文翻译

为了演示上面所有要点，让我们回到本章开头编写的那个简单包。你应记得，它的 `main.py` 文件会在文件夹被当作整体运行时自动执行，但它也可以扮演文件夹中被当作脚本启动的任意顶层文件的角色。上次见到这个文件时，它只包含一个 `print`；这里回顾一下我们即将使用的文件的最后状态，省得你翻回去：`python # 之前的 dir1/mod.py var = 'hack'print('Loading dir1.mod') # 之前的 dir1/main.py print('Executing dir1.main.py')`

深度理解

·核心概念：下面的实验将同一个 `main.py` 用四种方式启动，观察绝对导入与相对导入在不同启动协议下的成败——这是本章最容易踩坑的实际场景。

·底层实现：启动协议差异 = `sys.path` 与 `package` 的差异。直接命令行把文件所在目录设为 `home`；`-m` 则按包机制初始化包上下文。先记住这一点，下面三个实验都是它的推论。

·设计原因：同一文件要“既当程序、又当包”的两用性，是

Python 导入机制最微妙的设计权衡之一。

·实际问题：写命令行工具时，"直接运行报 `ModuleNotFoundError`、`-m` 却正常"或反之，是 Stack Overflow 高频问题，根因就在本节。

·初学者误区：以为"能不能 `import` 成功"只取决于文件在不在——其实还取决于谁在导入它（运行模式决定查找起点）。

24.18 普通导入热身 (The Normal-Import Warm-Up)

英文原文

This `main.py` worked as advertised, but things get more complicated if a file run as a top-level script tries to import another module in its own package folder—as in the rewrite of Example 24-7.

Example 24-7. `dir1/main.py` (modified)

```
python import mod print('Executing dir1.main.py:', mod.var.upper())
```

Coded this way, the file can be run with a direct command line (and both as a folder and a script), but not with Python's `-m` module-mode switch, which brings the package machinery online (notice the `init.py` output in this mode):

```
$ python3 dir1 # Launch with a direct command line
Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 dir1/main.py # As folder bundle or explicit  
file Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 -m dir1 # Launch with Python -m module  
switch Running dir1.init.py ModuleNotFoundError: N  
o module named'mod'
```

```
$ python3 -m dir1.main # As package root or explicit  
module Running dir1.init.py ModuleNotFoundError:  
No module named'mod'
```

The first two commands in this work because `main.py` is run as a normal, nonpackage program, and finds its imported sibling per the "home" entry on `sys.path`, the top-level file's own folder. The problem with the last two commands is that they use the file as part of a package: the `import mod` in `main.py` is then interpreted as an absolute import—which skips the enclosing package. Hence the fails.

中文翻译

这个 `main.py` 按预期工作，但如果一个被当作顶层脚本运行的文件试图导入自己包文件夹里的另一个模块，事情就变得复杂了——如示例 24-7 的改写所示。示例 24-7. `dir1/main.py` (修改版) `python import mod print('Executing dir1.main.py:', mod.var.upper())` 按这种方式编码，这个文件可以用直接命令行运行（既可作为文件夹、也可作为脚本），但不能用 Python 的 `-m` 模块模式开关运行——`-m` 会把包机制激活（注意该模式下出现了 `init.py` 的输出）：
\$ `python3 dir1` # 用直接命令行启动 Loading `dir1.mo`

d Executing dir1.main.py: HACK \$ python3 dir1/main.py # 作为文件夹整体或显式文件 Loading dir1.mod Executing dir1.main.py: HACK \$ python3 -m dir1 # 用 Python -m 模块开关启动 Running dir1.init.py ModuleNotFoundError: No module named 'mod' \$ python3 -m dir1.main # 作为包根或显式模块 Running dir1.init.py ModuleNotFoundError: No module named 'mod' 前两个命令能工作，是因为 main.py 被当作普通的、非包的程序运行，并根据 sys.path 上的 "home" 条目——即顶层文件自己的文件夹——找到了它要导入的兄弟模块。后两个命令的问题在于，它们把文件当作包的一部分来使用：main.py 中的 import mod 于是被解释为绝对导入——它会跳过所在包。因此失败。

深度理解

·核心概念：import mod 是绝对导入，绝对导入跳过所在包——这是两条规则的叠加效应，导致 -m 模式下失败。

·底层实现：-m dir1 时解释器执行 dir1/main.py，但此时模块被命名为 dir1.main（有包上下文），sys.path 中没有 dir1 目录本身（只有它的容器 dir0）；import mod 沿 sys.path 找顶层 mod，找不到。而直接运行时，模块名为 main，home = dir1 目录直接进 sys.path，import mod 就命中了 dir1/mod.py。

·设计原因：绝对导入“跳过包”是为了避免歧义——如果包内导入先搜自己，一个 import utils 究竟指自己还是指宿主的 utils 就无法区分了。干净语义是：想找自己，就显式写（from . import 或完整路径）。

·实际问题：python -m dir1 报 ModuleNotFoundError : No module named 'mod' 是最经典的"运行模式与导入方式不匹配"事故现场。

·初学者误区：以为 -m 与直接运行"只是写法不同"——它们对模块命名、sys.path 构成的影响完全不同，直接决定了 import 的成败。

代码分析

```
# dir1/__main__.py — 24-7
import mod                      # sys.path mod
print('Executing dir1.__main__.py:', mod.var.upper())
```

解释器如何处理：直接运行时，解释器把 main.py 作为 main 模块编译执行，同时把它的目录（dir1）作为 home 放进 sys.path——import mod 在 sys.path 里命中 dir1/mod.py，先打印 Loading dir1.mod（首次导入执行 mod 的顶层代码，var='hack'），再执行 print，输出 HACK。-m dir1 时，解释器以 dir1.main 为模块名，先执行 dir1/init.py（打印 Running dir1.init.py），而 sys.path 顶层没有 mod——于是 ModuleNotFoundError。可见：同一行 import 代码，成败完全取决于运行协议注入的模块名与搜索路径。

24.19 相对导入的冒险 (The Relative-Import Adventure)

英文原文

Our first reaction might be to add relative imports to appease the package system—as in Example 24-8.

Example 24-8. `dir1/main.py` (modified)

```
python from . import mod print('Executing dir1.main.py:', mod.var.upper())
```

Coded this way, we've fixed `-m` usage, but broken direct command lines:

```
$ python3 -m dir1 Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 -m dir1.main Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 dir1 ImportError: attempted relative import with no known parent package
```

```
$ python3 dir1/main.py ImportError: attempted relative import with no known parent package
```

The first two commands in this work because `-m` invokes package behavior, which enables the relative `"."` import syntax that searches the package and finds the target module. The problem with the last two commands is that Python doesn't allow relative imports to be used in nonpackage mode—which dooms these runs from the start, regardless of search-path settings. Hence the fails.

Under this regime, then, it would seem we must choose package or nonpackage roles for our code.

中文翻译

我们的第一反应可能是添加相对导入来讨好包系统——如示例 24-8 所示。示例 24-8. dir1/main.py (修改版) `python from . import mod print('Executing dir1.main.py:', mod.var.upper())` 按这种方式编码，我们修好了 `-m` 用法，却弄坏了直接命令行：`$ python3 -m dir1 Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK $ python3 -m dir1.main Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK $ python3 dir1 ImportError: attempted relative import with no known parent package $ python3 dir1/main.py ImportError: attempted relative import with no known parent package` 前两个命令能工作，是因为 `-m` 调用了包行为，激活了相对 `"."` 导入语法——它搜索包并找到了目标模块。后两个命令的问题在于：Python 不允许相对导入在非包模式下使用——这从一开始就注定了这两次运行失败，与搜索路径设置无关。因此失败。在这种制度下，看来我们必须要在“包”与“非包”两种角色之间为代码做出选择。

深度理解

·核心概念：相对导入与绝对导入是互斥的两难：修好 `-m` 就弄坏直接运行，反之亦然。这正是“全有或全无”权衡的现场演示。

·底层实现：直接运行时，`main.py` 的 `package` 为空字符串（顶层脚本没有包上下文），编译器/导入系统无法解析 `from . import mod` 里的 `"."` 指向哪里，直接抛 `ImportError: attempted relative import with no known parent`

package——错误信息直译就是"在找不到父包的情况下尝试了相对导入"。

·设计原因：顶层脚本没有"父包"概念（它可能来自任何目录），所以相对导入对它必须非法——否则指向不明。这是语义自治的必然结果，不是 bug。

·实际问题：你写的包若坚持用相对导入，用户就只能用 `-m` 启动它；想"双击就能跑"就得另想办法。python dir1 会得到一句很吓人的 `ImportError`，初学者常误以为文件损坏。

·初学者误区：以为 `from . import mod` "应该"在任何情况下都能找到同目录的 `mod`——错，它只在包上下文存在时有效；也以为改 `PYTHONPATH` 能救活它——与搜索路径无关，是模式问题。

代码分析

```
# dir1/__main__.py — 24-8
from . import mod                # dir1
print('Executing dir1.__main__.py:', mod.var.upper())
```

解释器如何处理：python -m dir1 时，解释器为 `main.py` 建立包上下文（`package = 'dir1'`），`from . import mod` 解析为"在 `dir1.path` 中找 `mod`"，成功加载并打印 `Loading dir1.mod`。python dir1 时，解释器把文件当作裸顶层脚本（`package = ''`），`from . import mod` 中"."`无处可指——在编译阶段之后、解析阶段立即抛出 ImportError: attempted relative import with no known parent package。注意此时连 mod 的加载代码都不会执行，因为错误发生在定位阶段。`

24.20 绝对导入的解法 (The Absolute-Import Solution)

英文原文

As noted, though, an easy way out of this constraint is to use normal package imports that explicitly spell out absolute import paths (which, again, are actually relative to `sys.path`) from the package root—as in Example 24-9.

Example 24-9. `dir1/main.py` (modified)

```
python import dir1.mod print('Executing dir1.main.py :', dir1.mod.var.upper())
```

As also noted, the package root, `dir1`, will normally be on `sys.path`, because that's the only way to use its code from outside the package (clients must import with paths that start at the `dir1` package root too).

To demo here, we'll add the package root explicitly using a relative—in filesystem terms—path of `"."` for the current directory (in typical use, this might instead be an absolute path, or Python's automatic site-packages root of installed packages). We must set this here because the top-level file's "home" on the search path in direct-command mode is one level below the package root, and both direct-command and `-m` modes need access to the package root in general:

```
$ export PYTHONPATH=. # Or similar outside Unix: see Chapter 22
$ python3 dir1 Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 dir1/main.py Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 -m dir1 # Both usage modes work, sans "."
imports Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK
```

```
$ python3 -m dir1.main Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK
```

Now launches by both direct command lines and the `-m` switch work, because we're not using a tool that limits the utility of our code to just one mode. The end result is dual-mode code—it can be used as both program and package.

If you'd rather not repeat import paths at each item reference, both launch modes also work if we code `main.py` in any of these ways (test on your own to verify; this is just import norms at work):

```
python import dir1.mod as mod print('Executing dir1.main.py:', mod.var.upper())
from dir1 import mod print('Executing dir1.main.py:', mod.var.upper())
from dir1.mod import var print('Executing dir1.main.py:', var.upper())
```

For more fun, you can also use a `from` in this scheme, if you add an `__all__` to the package root's initialization file (subject to all the standard disclaimers about

ut the evils of from outlined in Chapter 23):

```
python # dir1/init.py all = ['mod'] # dir1/main.py from
m dir1 import print('Executing dir1.main.py:', mod.var
.upper())
```

Again, if you're sure that your code will only ever be used as part of a package, you could use relative imports in all of its files to access same-package modules. Moreover, the absolute-path solution arrived at here could be applied only for files meant to be launched as top-level scripts; other files only imported can use package-relative imports.

In both cases, relative imports come with the advantage that a same-named module earlier on `sys.path` won't accidentally override a crucial module in the package. But they also limit a file to package-only roles; if you want package files to support both launches and imports, relative imports are not your best option.

It should also be noted that we're skipping some details here for space. For one, `-m` mode unusually also checks the CWD for absolute imports, making the demo's `PYTHONPATH` setting optional for this mode and demo only. For another, some other resources suggest that relative-import failures in scripts stem from their name `'main'`, but files run with `-m` have the same name and employ other protocols that are too obscure to cover here.

Because package-relative imports are both prone to change and mostly meant for programmers building

large-scale libraries of code for others to use, this Python-learners text will defer to Python's docs for further details. Modules are feature-rich tools, to say the least, but the good news is that namespace packages—the topic of the next and final section of this chapter—do not add any new syntax, but simply allow a module to span folders. To see how, let's move on.

中文翻译

不过正如前面所说，跳出这个约束的简单办法是：使用普通的包导入，显式写出从包根开始的“绝对”导入路径（再说一次，它们其实也是相对于 `sys.path` 的）——如示例 24-9 所示。示例 24-9. `dir1/main.py` (修改版) `python import dir1.mod print('Executing dir1.main.py:', dir1.mod.__var.upper())` 同样如前所述，包根 `dir1` 通常会在 `sys.path` 上——因为这是包外代码使用它的唯一途径（客户方也必须用从 `dir1` 包根开始的路径导入）。为了演示，我们显式把包根加进路径——在文件系统意义上用相对路径 `."` 指当前目录（实际使用中，这可能是绝对路径，也可能是 Python 自动生成的、已安装包的 `site-packages` 根）。这里必须设置它，是因为直接命令行模式下，顶层文件的“home”在搜索路径上位于包根下一层，而直接命令行与 `-m` 两种模式一般都需要能访问到包根：`$ export PYTHONPATH=.` # 或 Unix 之外平台的类似做法：见第 22 章

```
$ python3 dir1 Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK $ python3 dir1/main.py Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK $ python3 -m dir1 # 两种使用模式都能工作，无需
```

." 导入Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK \$ python3 -m dir1.main Running dir1.init.py Loading dir1.mod Executing dir1.main.py: HACK 现在，直接命令行与 -m 开关两种启动方式都能工作，因为我们没有使用那种把代码用途限制在单一模式下的工具。最终结果是双模式代码——既可以当程序用，也可以当包用。如果你不想在每次引用条目时都重复导入路径，用下面任何一种方式编写 main.py，两种启动模式也都能工作（请自行验证；这只是导入规范在起作用）：
`python import dir1.mod as mod print('Executing dir1.main.py:', mod.var.upper())`
`from dir1 import mod print('Executing dir1.main.py:', mod.var.upper())`
`from dir1.mod import var print('Executing dir1.main.py:', var.upper())`
更有趣的是，如果你给包根的初始化文件加上 all，这个方案里还能用 from（当然要受第 23 章关于 from 之恶的种种标准警告约束）：
`python # dir1/init.py all = ['mod']`
`# dir1/main.py from dir1 import print('Executing dir1.main.py:', mod.var.upper())`
再说一次，如果你确信自己的代码只会作为包的一部分被使用，那么可以在它的所有文件中都用相对导入来访问同包模块。而且，这里得出的"绝对路径"解法可以只用于那些打算作为顶层脚本启动的文件；其他仅被导入的文件可以继续使用包相对导入。两种情况下，相对导入都有"sys.path 上更靠前的同名模块不会意外覆盖包内关键模块"这一优势。但它们也把文件限制为仅包内角色；如果你希望包文件同时支持启动和导入两种用法，相对导入不是你的最佳选择。还应当指出，为节省篇幅我们略过了一些细节。其一，-m 模式非同寻常地也会检查 CWD 进行绝对导入，这使演示中的 PYTHONPATH 设置在本

模式、本演示中变为可选。其二，有些资料声称脚本中相对导入失败源于其名为'main'，但用 -m 运行的文件也是这个名字，却采用了其他此处过深不便详述的协议。因为包相对导入既容易变化，又主要面向那些为他人构建大规模代码库的程序员，所以这本面向 Python 学习者的书将把更多细节留给 Python 官方文档。模块绝对是特性丰富的工具，但好消息是：命名空间包——本章最后一个小节的主题——不添加任何新语法，只是让一个模块可以横跨文件夹。想看看怎么做，让我们继续。

深度理解

- 核心概念：完整路径导入 (`import dir1.mod`) 是同时兼容"程序模式"与"包模式"的通用解——但要求包根 (`dir1`) 能在 `sys.path` 上被找到，否则两种模式都得补路径配置。

- 底层实现：`export PYTHONPATH=.` 把当前目录 (`dir0`，`dir1` 的容器) 加进 `sys.path`，于是 `import dir1.mod` 在两种模式下都能定位到 `dir1/mod.py`。注意直接运行 `python3 dir1` 时也打印了 `Running dir1.init.py`——因为导入 `dir1.mod` 会先触发包 `dir1` 的初始化，这是"路径导入触发包机制"的连锁反应，与启动方式无关。

- 设计原因：这是 Python 社区总结出的"双模式"最佳实践：库代码用相对导入（防污染），入口脚本用完整路径导入（双模式兼容）。两者各司其职。

- 实际问题：把"可执行入口"放进包并同时支持 `python dir1` 与 `python -m dir1`（如 `python -m http.server`、`python -m pip` 这类 CLI 模式），本质都是本节方案的工程

化。

·初学者误区：以为 `from . import mod` 失败是"Python 的 bug"或"路径没配好"——其实是模式不匹配；也以为 `PYTHONPATH` 是万能的——它只是把包根"翻译"给绝对导入，救不了相对导入在非包模式下的硬性失败。

代码分析

```
# dir1/__main__.py — 24-9
import dir1.mod                                # dir1
print('Executing dir1.__main__.py:', dir1.mod.var.upper())

# as
import dir1.mod as mod
print('Executing dir1.__main__.py:', mod.var.upper())

# from
from dir1 import mod
print('Executing dir1.__main__.py:', mod.var.upper())

# from __all__ from *
# dir1/__init__.py:
__all__ = ['mod']                                # from dir1 import * mod
# dir1/__main__.py:
from dir1 import *
print('Executing dir1.__main__.py:', mod.var.upper())
```

解释器如何处理：python dir1 时 home 是 dir1 目录（比包根低一层），python -m dir1 时 sys.path 也不含 dir1——两种模式单独看都找不到 dir1，于是实验先 export PYTHONPATH=. 把容器目录放上搜索路径。之后 import dir1.mod 在任何模式下都先加载 dir1（打印 Running dir1.init.py）、再加载 mod.py（打印 Loading dir1.

mod) , 最后执行 print。from dir1 import 时, 解释器读取 dir1.init.py 里的 all = ['mod'], 按列表导入 mod 子模块并绑定, 于是 mod.var 可用——all 在这里是"from 白名单"。

24.21 命名空间包 (Namespace Packages)

英文原文

Now that you've learned all about package and package-relative imports, there's one last package-related subject to cover. The evolutionary path of modules in Python eventually led to what are known as namespace packages—packages that may be composed of one or more folders located on different parts of the module search path. While packages split across folders are probably atypical in the wild, the generalizations added to support namespace packages in the import-search algorithm (procedure) also allow for packages without init.py files in general. Since this revised algorithm is now just the import search, namespace packages are something of an incidental topic. To understand namespace packages' place in the broader modules picture, though, as well as the changes they ushered in, we need a quick history lesson.

中文翻译

现在你已经学完了包导入和包相对导入，还剩最后一个与包相关的主题要讲。Python 模块的演进之路最终通向所谓的命名空间包（namespace packages）——可能由一个或多个、位于模块搜索路径不同位置的文件夹组成的包。虽然跨文件夹拆分的包在实际中可能并不典型，但为支持命名空间包而对导入搜索算法（过程）所做的泛化，同时也让“没有 `init.py` 文件的包”在一般情况下成为可能。既然这个修订后的算法现在就是导入搜索本身，命名空间包在某种程度上成了一个“顺带”的主题。不过，要理解命名空间包在更广阔的模块图景中的位置、以及它们带来的变化，我们需要上一堂简短的历史课。

深度理解

- 核心概念：命名空间包 = “可能跨越多个文件夹的包”，且不需要 `init.py`。它不是一个新语法，而是导入搜索算法的一个泛化结果。
- 底层实现：path 是理解它的钥匙——常规包只有一个目录，命名空间包的 path 是目录列表，导入系统在列表内逐一搜索。
- 设计原因：PEP 420 (Python 3.3) 之前，多目录合并包只能靠各自为政的 hack (`.pth` 技巧等)；统一算法后，`init.py` 从“必需”降级为“可选”。
- 实际问题：大库拆成多个独立分发单元（如 `pkg.foo`、`pkg.bar` 分别安装）时，命名空间包让它们共享同一个 `pkg` 名字。

·初学者误区：以为命名空间包是"全新的导入方式"——它仍然相对于 `sys.path`，只是允许"同名多目录合并"。

24.22 Python 的导入模型 (Python Import Models)

英文原文

All told, Python has four distinct import schemes. The following enumerates them from original to newest, with representative but incomplete examples (as we've seen, from also allows a wildcard and reload reloads any module passed to it, but these are just add-on topics):

- Basic modules — The original model, used to import files and their contents, relative to the `sys.path` module search path: `python import module from module`
- Basic packages — The original package model used to import from folder paths relative to the `sys.path` search path, where each package is contained in a single directory that has an `init.py` file: `python import folder.folder.module from folder.module`
- Package-relative imports — The model used for intrapackage (same-package) imports of the prior section, with its relative and absolute lookup rules for imports with and without leading dots, respectively: `python from . import module from .module import name`
- Namespace packages — The newest package model, which is still relative to `sys.path`, but allo

ws packages to span multiple directories, and removes the requirement that packages must define `init.py` files: `python import anyfolder.anyfolder.module` from `anyfolder` import name The first two of these models are self-contained, but the third tightens up the search order and extends syntax for same-package imports, and the fourth upends some of the notions and requirements of the prior package model. In fact, Python now formally defines two flavors of packages:- The original model, now known as regular packages- The alternative model, known as namespace packages The original and alternative package models are not mutually exclusive and can be used simultaneously in the same program. In fact, the namespace package model works as something of a fallback option, recognized only if basic modules and regular packages of the same name are not present on the module search path. Despite their showy title, namespace packages are really just a mutation of import search, as the following sections explain.

中文翻译

总计起来，Python 有四种不同的导入方案。下面按从最早到最新的顺序列举，并给出有代表性但不完整的例子（正如我们所见，`from` 还支持通配符，`reload` 能重载传给它的任何模块，但这些只是附加话题）：基本模块（Basic modules）——最早的模型，用于导入文件及其内容，相对于 `sys.path` 模块搜索路径：`python import module from mod`

ule import name 基本包 (Basic packages) ——最早的包模型，用于从相对于 sys.path 搜索路径的文件夹路径导入，其中每个包都位于一个带 init.py 文件的单一目录中：python import folder.folder.module from folder.module import name 包相对导入 (Package-relative imports) ——用于上一节所述包内 (同包) 导入的模型，分别对带与不带前导点号的导入规定了相对和绝对的查找规则：python from . import module from .module import name 命名空间包 (Namespace packages) ——最新的包模型，仍然相对于 sys.path，但允许包横跨多个目录，并取消了"包必须定义 init.py 文件"的要求：python import anyfolder.anyfolder.module from anyfolder import name 前两种模型是自足的，第三种收紧了搜索顺序、扩展了同包导入的语法，第四种则颠覆了先前包模型的某些观念和要求。事实上，Python 现在正式定义了两种包：- 最早的模型，现在称为常规包 (regular packages) - 替代模型，称为命名空间包 (namespace packages) 这两种包模型并不互斥，可以在同一个程序中同时使用。事实上，命名空间包模型扮演的是某种后备选项 (fallback option) 的角色——只有当模块搜索路径上不存在同名的基本模块和常规包时，它才会被识别。尽管名头响亮，命名空间包其实只是导入搜索的一次变异，后面几节会解释这一点。

深度理解

·核心概念：四种导入模型层层演进，命名空间包是"导入搜索的变异"而非新语法。当前 Python 中所有模型同时有效。

- 底层实现：四种模型最终共享同一个 importlib 导入管线；区别只在于候选查找集 (sys.path vs 包 path) 与包判定规则 (是否有 init.py)。
 - 设计原因：兼容性至上——Python 不能删掉旧模型，只能在搜索算法里加"后备"分支，让新旧代码共存。
 - 实际问题：面试题常问"Python 有几种包"——答案：常规包与命名空间包，两种模型；而"导入模型"演进史 (文件 → 单目录包 → 相对导入 → 命名空间包) 正是理解 sys.path/path 的最佳脉络。
 - 初学者误区：以为命名空间包"更新、更好"该优先用——实际上常规包仍是主流 (有初始化钩子、优先级更高)，命名空间包只是后备与拆分场景的工具。
-

24.23 命名空间包的理由 (Namespace-Package Rationales)

英文原文

First off, it's important to know that a namespace package is not fundamentally different from a regular package: it is just a different way of creating packages. Moreover, they are still relative to sys.path at the top level: the leftmost component of a dotted namespace-package path must still be located in an entry on the normal module search path. In terms of physical structure, though, regular and namespace packages have notable differences. Regular packages have an ini

t.py file that is run automatically, and they reside in a single directory. Namespace packages, by contrast, cannot contain an `init.py`, and may or may not span multiple directories collected at import time. Because the presence of `init.py` differentiates package type, none of the directories that make up a namespace package can have this file, but the content nested within each of them is treated as a single composite package. The rationale for namespace packages is rooted in package installation goals that may seem obscure unless you are responsible for such tasks. In short, though, they resolve a potential for collision of multiple `init.py` files when package parts are merged, by removing this file completely. Moreover, by providing standard support for packages that can be split across multiple directories and located in multiple `sys.path` entries, namespace packages both enhance install flexibility and replace multiple incompatible solutions that had arisen to address this goal. Though split-folder packages have some narrow roles, average Python users will probably find namespace packages' biggest benefit to be that they remove the requirement for package `init.py` initialization files, and thus allow any directory of code to be used as an importable package. To see how, let's move on to the nitty-gritty of import search.

中文翻译

首先，重要的一点是：命名空间包与常规包并没有本质区别——它只是创建包的另一种方式。而且，它们在顶层仍然是相对于 `sys.path` 的：点号命名空间包路径最左边的组件，仍必须位于普通模块搜索路径的某个条目中。不过在物理结构上，常规包与命名空间包有明显的差异。常规包有一个自动运行的 `init.py` 文件，且位于单一目录中。相比之下，命名空间包不能包含 `init.py`，并且可能（也可能不）横跨多个在导入时收集起来的目录。因为 `init.py` 的存在区分了包类型，构成命名空间包的每个目录都不能有这个文件，但它们各自嵌套的内容会被当作一个统一的复合包来处理。命名空间包的理由植根于包安装的目标——除非你负责这类工作，否则可能觉得它晦涩难懂。简而言之，它通过彻底移除 `init.py` 文件，解决了多个 `init.py` 文件在包部件合并时可能发生的冲突。此外，通过为标准支持"可拆分到多个目录、位于多个 `sys.path` 条目中"的包提供官方方案，命名空间包既增强了安装灵活性，也取代了为实现这一目标而冒出来的多种互不兼容的解决方案。虽然跨文件夹拆分的包只有一些狭窄的应用场景，普通 Python 用户会发现命名空间包最大的好处是：它移除了包 `init.py` 初始化文件的要求，从而让任何代码目录都可以作为可导入的包使用。想看看具体怎么做，让我们深入导入搜索的细节。

深度理解

·核心概念：`init.py` 的有无 = 包类型的判据（有 → 常规包；无 → 命名空间包）。命名空间包是"多目录虚拟合并"的官方方案。

- 底层实现：安装时合并多个分发单元（如 `pip install pkg-a pkg-b` 各自带 `pkg/` 子目录）时，若都带 `init.py`，后装的会覆盖先装的（文件冲突）；去掉这个文件，两个 `pkg/` 目录就能和平共存于 `sys.path` 不同条目中。
 - 设计原因：PEP 420 的动机来自生态需求——大项目（如 `zope`、`scikit-learn` 生态）拆分成独立可安装的命名空间包，需要官方合并语义，而不是各写各的 `.pth hack`。
 - 实际问题：对普通开发者，最大的实际收益是“任何目录都能 `import`”——不必为了“把目录变成包”而被迫加一个空文件。
 - 初学者误区：以为“目录里没有 `init.py` 就不能导入”——那是 Python 3.2 及之前的旧规则；3.3+ 之后普通目录自动成为命名空间包，只是优先级低于常规包。
-

24.24 模块搜索算法 (The Module Search Algorithm)

英文原文

To understand the way that namespace packages change and extend import search, we have to look under the hood to see how the import operation works. As we've seen, Python fulfills imports by searching for a name among a set of candidate folders. For the leftmost components of imports, the set of candidates is all the directories listed on the `sys.path` module search path. For components of packages either nested in p

ackage imports or named in relative imports, the set of candidates is just the package itself. While package folders have just one instance of a given name, search paths may have many. The way the import search selects items from these candidates is subtler than implied so far. For each directory in an import search's set of candidates, Python tests for a variety of matches to an imported name, in the following order (using Unix / for folder separators and {...} to mean a choice): 1. If directory/name/init.py is found, a regular package is imported and returned. 2. If directory/name.{py, pyc, or other module extension} is found, a simple module is imported and returned. 3. If directory/name is found and is a directory, it is recorded and the scan continues with the next directory in the search's set of candidates. 4. If none of the above was found, the scan continues with the next directory in the search's set of candidates. If this search's candidate scan completes without returning a regular package or module by steps 1 or 2, and at least one directory was recorded by step 3, then a namespace package is created and returned. Else an error is reported. The creation of the namespace package happens immediately and is not deferred until a lower-level import occurs. The new namespace package is a module object that does not have a file attribute, but has a path set to an iterable of the directory path strings that were found and recorded during the candidates scan by step 3. T

his path attribute is then used as the set of candidate s for later and deeper accesses, to search the one or more component folders of the namespace package. That is, each recorded entry on a namespace package 's path is searched whenever further-nested items are requested, instead of the sole directory of a regular package. Viewed another way, the path attribute of a namespace package serves the same role for lower-level components that `sys.path` does at the top for the leftmost component of package import paths; it becomes the "parent" search path for accessing lower items using the same four-step procedure just sketched . The net result is that a namespace package is a sort of virtual concatenation of directories located on one or more search candidates. Once a namespace package is created, though, there is no functional difference between it and a regular package; it supports everything we've learned for regular packages, including package-relative import syntax. Importantly, because a single directory lacking an `init.py` but nested in a search-candidate folder is classified as a namespace package by this algorithm's step 3, any such directory qualifies as a package. The only operational difference between such a single folder and a regular package folder is that the former has lower search precedence: both same-named folders with an `init.py` and simple modules anywhere in a search path are chosen first by steps 1 and 2. In other words, although the mods ma

de to support namespace packages make `init.py` files optional in package folders, adding one, even if empty, ensures that a package will be selected instead of a same-named folder later on a search path (it may also boost import speed by ending search-path scans at step 1, but this is a one-time event). Whether this, or the other roles of `init.py` we met earlier, warrants including this file will naturally depend on your goals.

中文翻译

要理解命名空间包如何改变和扩展导入搜索，我们必须掀开引擎盖，看看导入操作是怎么工作的。如我们所见，Python 通过在一组候选文件夹中搜索某个名字来完成导入。对于导入最左边的组件，候选集就是 `sys.path` 模块搜索路径上列出的所有目录；对于包导入中嵌套的、或在相对导入中被点名的包组件，候选集就只是包本身。包文件夹对某个名字只有一个实例，而搜索路径上可能有多。导入搜索从这些候选中挑选条目的方式，比迄今为止暗示的要微妙。对于导入搜索候选集中的每个目录，Python 按以下顺序测试与导入名字的多重匹配（用 Unix 的 `/` 表示文件夹分隔符，用 `{...}` 表示“任选其一”）：

1. 如果找到 `directory/name/init.py`，就导入并返回一个常规包。
2. 如果找到 `directory/name.{py, pyc, 或其他模块扩展名}`，就导入并返回一个简单模块。
3. 如果找到 `directory/name` 且它是一个目录，就记录下来，然后继续扫描候选集中的下一个目录。
4. 如果以上都没有找到，就继续扫描候选集中的下一个目录。

如果这次搜索的候选扫描结束，既没有在第 1、2 步返回常规包或模块，且第 3 步至少记录了一个目录，那么就创建并返

回一个命名空间包。否则报告错误。命名空间包的创建是立即发生的，不会推迟到更低层级的导入出现时才进行。新的命名空间包是一个没有 file 属性、但 path 被设为"第 3 步候选扫描中记录下来的目录路径字符串的可迭代对象"的模块对象。随后，这个 path 属性被用作后续更深层访问的候选集，用于搜索命名空间包的一个或多个组件文件夹。也就是说，每当请求更嵌套的条目时，命名空间包 path 上记录的每个条目都会被搜索——而不是像常规包那样只有唯一一个目录。换一个角度看：命名空间包的 path 属性对更低层组件所起的作用，正如 sys.path 在顶层对包导入路径最左组件所起的作用；它成为访问更低层条目时的"父级"搜索路径，同样使用上面勾画的四步流程。最终结果是：命名空间包是一个或多个搜索候选上目录的某种虚拟拼接。不过，一旦命名空间包创建完成，它与常规包之间就没有功能上的差别了；它支持我们为常规包学到的所有特性，包括包相对导入语法。重要的是，由于"嵌套在搜索候选文件夹中、且缺少 init.py 的单个目录"会被本算法的第 3 步归类为命名空间包，任何这样的目录都有资格充当包。这种单目录与常规包文件夹之间唯一的操作性差异是：前者的搜索优先级更低——搜索路径上任何地方的同名文件夹（带 init.py 的）和简单模块，都会先被第 1、2 步选中。换句话说，虽然为支持命名空间包所做的修改使 init.py 文件在包文件夹中变为可选，但添加一个（哪怕是空的）能确保：当搜索路径上更靠后存在同名文件夹时，你的包会被选中（它还可能通过在第 1 步就结束搜索路径扫描来提升导入速度，但这是一次性事件）。至于要不要包含这个文件，自然取决于你的目标——无论是这个作用，还是我们前面见到的 init.py 的其他作用。

深度理解

·核心概念： 四步搜索算法是本章的"发动机舱"。要点： 常规包 > 简单模块 > 目录记录（候选）——命名空间包只在所有候选都扫完仍无命中时诞生。

·底层实现： 解释器对 `sys.path` 每个条目依次测试目录/名字的二态：带 `init.py` 的目录（常规包）、模块文件、无 `init.py` 的目录（记录）。扫完后若"只有目录记录"，就用它们拼成 `NamespacePath`，创建模块对象；`file` 缺席而 `path` 是列表——这是命名空间包在 `repr` 中显示 (`namespace`) 的原因。

·设计原因：为什么第 3 步"先记录、继续扫"？因为后面可能还有常规包或模块——优先级要留给它们；又为什么"至少一个目录"就能成包？为了允许跨 `sys.path` 多个条目的合并（虚拟拼接）。

·实际问题：空 `init.py` 的实际价值——同名目录冲突时（比如 `site-packages` 里别人装了个同名包），有 `init.py` 的一方稳赢；空文件也顺带让导入在第 1 步就收敛，省得继续扫描。

·初学者误区：以为"名字相同的多个目录"会报错或互相覆盖——只要都无 `init.py`，它们会被合并成一个命名空间包（虚拟拼接），内容共存。

代码分析

```

# name dir
for dir in candidates:                                # sys.path __pa...
    if exists(dir/name/__init__.py):                  # ①
        return load_regular_package()
    if exists(dir/name.py or .pyc or .so ...):         # ②
        return load_module()
    if isdir(dir/name):                                # ③
        recorded.append(dir/name)

#
if recorded:
    return create_namespace_package(recorded)         # __path__ =...
else:
    raise ModuleNotFoundError(name)

```

解释器如何处理：这段逻辑对应 `importlib.bootstrap` 中 `findandload` 的查找阶段。注意第 3 步只是记录而不返回——一个“无 `init.py` 的目录”永远不能立刻胜出，它必须等到所有候选都扫描完、且没有常规包/模块命中时才有机会。创建命名空间包时，模块对象没有 `file`（它没有一个真实文件），`path` 被赋为 `NamespacePath` 对象（内部是记录的目录字符串列表）；此后导入 `ns.sub` 时，候选集换成这个列表，对其中每个目录重跑四步流程——于是 `part1/sub/mod1` 与 `part2/sub/mod2` 能在同一个 `sub` 名下共存。

24.25 命名空间包实战 (Namespace Packages in Action)

英文原文

Technically, we've already seen namespace packages at work: the opening step of the demo at the start of this chapter made single-folder namespace packages because its folders didn't yet have `init.py` files. Again, any folder nested in a `sys.path` search-path folder qualifies as a package, as long as it's not hidden by a same-named "regular" package with `init.py` or a simple module elsewhere on the path.

To see the grander folder-concatenation effect of "virtual" namespace packages work, let's work through a quick demo. To begin, make two modules in a nested directory structure that has subdirectories named `sub` located in different parent directories, `part1` and `part2`—like this in indentation notation meant to represent nesting:

```
ns/ part1/ sub/ mod1.py part2/ sub/ mod2.py
```

Note that there are no `init.py` files here—as noted earlier, these files cannot be used in namespace packages, as this is their chief differentiation from regular, single-folder packages. We can create this demo's folders with console commands like the following; translate `/` to `\` and omit the `-p` on Windows, or use a file explorer or the prebuilt folders in the examples package if you prefer:

```
$ mkdir -p ns/part1/sub # Two subdirs of same nam
```

e in different dirs \$ mkdir -p ns/part2/sub # And similar on Windows The modules at the end of these paths are coded in Examples 24-10 and 24-11 to simply print on imports as a trace.

Example 24-10. ns/part1/sub/mod1.py

```
python print('Loading ns/part1/sub/mod1')
```

Example 24-11. ns/part2/sub/mod2.py

```
python print('Loading ns/part2/sub/mod2')
```

Now, if we add both part1 and part2 to the module search path, sub becomes a namespace package spanning both folders, with the two module files available under that name even though they live in separate physical directories. Here's the path-setting command on Unix (for Windows, use set and;, and see Chapter 22 for more tips); this uses paths relative to the CWD, but in practice would more likely list two full, absolute paths instead:

```
$ export PYTHONPATH=ns/part1:ns/part2
```

When imported directly, the namespace package is the virtual concatenation of its individual directory components, and allows further nested parts to be accessed through its single, composite name with normal imports (as usual, paths have been shortened here for space with "... " and line breaks were added for fit):

```
$ python3 >> import sub >> sub # Namespace packages: nested search paths <module'sub' (namespace)
from ['../LP6E/Chapter24/ns/part1/sub', '../LP6E/Ch
```

```
apter24/ns/part2/sub']> >> from sub import mod1
Loading ns/part1/sub/mod1 >> import sub.mod2 # Content from two different directories
Loading ns/part2/sub/mod2 >> mod1 <module'sub.mod1' from '/.../LP6E/Chapter24/ns/part1/sub/mod1.py'> >> sub.mod2 <module'sub.mod2' from '/.../LP6E/Chapter24/ns/part2/sub/mod2.py'>
```

This split-folder package also works if we import through the namespace package name immediately—because the namespace package is made when first reached, the timing of path extensions is irrelevant:

```
$ python3 >> import sub.mod1 Loading ns/part1/sub/mod1 >> import sub.mod2 # One package spanning two directories
Loading ns/part2/sub/mod2 >> sub.mod1 <module'sub.mod1' from '/.../LP6E/Chapter24/ns/part1/sub/mod1.py'> >> sub.mod2 <module'sub.mod2' from '/.../LP6E/Chapter24/ns/part2/sub/mod2.py'> >> sub <module'sub' (namespace) from ['/.../LP6E/Chapter24/ns/part1/sub','/.../LP6E/Chapter24/ns/part2/sub']> >> sub.path NamespacePath(['/.../LP6E/Chapter24/ns/part1/sub','/.../LP6E/Chapter24/ns/part2/sub'])
```

Interestingly, relative imports work in namespace packages too, if we add one as in Example 24-12.

Example 24-12. ns/part1/sub/mod1.py (modified)

```
python from . import mod2 print('Loading ns/part1/sub/mod1')
The added package-relative import state
```

ment references a file in the package—even though the referenced file resides in a different directory:

```
$ python3 >> import sub.mod1 # Relative import of  
mod2 in another dir Loading ns/part2/sub/mod2 Loa  
ding ns/part1/sub/mod1 >> import sub.mod2 # Alre  
ady imported by mod1.py: not rerun >> sub.mod2 <  
module'sub.mod2' from'../LP6E/Chapter24/ns/part2  
/sub/mod2.py'>
```

As you can see, namespace packages are like ordinary single-directory packages in every way, except for having a split physical storage. By extension, this is why code folders without `init.py` files are exactly like regular packages, but with no initialization logic to be run; they're just an instance of namespace packages with a single directory.

Like package-relative imports, this book is also going to defer to Python's manuals for more details on this subject. Namespace packages are a potentially useful tool, but most Python learners are probably better served by first mastering packages that map to real folders, before tackling those that are spread virtually across a host's directories.

中文翻译

严格来说，我们已经见过命名空间包在工作了：本章开头的演示在起步阶段就做出了单文件夹命名空间包——因为当时它的文件夹还没有 `init.py` 文件。再说一次：只要没有被

搜索路径上别处的、带 `init.py` 的同名"常规"包或简单模块"压住"，任何嵌套在 `sys.path` 搜索路径文件夹里的目录都有资格充当包。为了看到"虚拟"命名空间包那种更宏大的文件夹拼接效果，让我们快速过一遍演示。首先，在一个嵌套目录结构中制作两个模块，让名为 `sub` 的子目录位于不同的父目录 `part1` 和 `part2` 之下——用缩进记号表示嵌套如下：

```
ns/ part1/ sub/ mod1.py part2/ sub/ mod2.py
```

注意这里没有任何 `init.py` 文件——如前所述，命名空间包中不能使用这些文件，这正是它与常规单目录包的主要区别。我们可以用下面这样的控制台命令创建本演示的文件夹；Windows 上把 `/` 换成 `\` 并去掉 `-p`，或者如果你愿意，也可以用文件资源管理器或示例包中预建的文件夹：

```
$ mkdir -p ns/part1/sub # 两个同名子目录位于不同目录$ mkdir -p ns/part2/sub # Windows 上类似这些路径末端的模块在
```

示例 24-10 和 24-11 中编写，导入时只打印一行作为追踪。

```
示例 24-10. ns/part1/sub/mod1.py python print('Loading ns/part1/sub/mod1')
示例 24-11. ns/part2/sub/mod2.py python print('Loading ns/part2/sub/mod2')
```

现在，如果我们把 `part1` 和 `part2` 都加入模块搜索路径，`sub` 就成为一个横跨两个文件夹的命名空间包，两个模块文件虽然住在不同的物理目录里，却都能在这个名字下使用。下面是 Unix 上设置路径的命令（Windows 上用 `set` 和分号，更多技巧见第 22 章）；这里用的是相对 CWD 的路径，实际使用中更可能列出两个完整绝对路径：

```
$ export PYTHONPATH=ns/part1:ns/part2
```

直接导入时，命名空间包就是它各个目录组件的虚拟拼接，并允许通过它单一的复合名字、用普通导入访问更深层的部件（和往常一样，路径为了排版被缩写成"`...`"并加了换行）：

```
$ python3 >>> im
```

```

port sub >>> sub # 命名空间包：嵌套的搜索路径<module'sub' (namespace) from ['../../LP6E/Chapter24/ns/part1/sub','../../LP6E/Chapter24/ns/part2/sub']> >>> from sub import mod1 Loading ns/part1/sub/mod1 >>> import sub.mod2 # 内容来自两个不同的目录Loading ns/part2/sub/mod2 >>> mod1 <module'sub.mod1' from'../../LP6E/Chapter24/ns/part1/sub/mod1.py'> >>> sub.mod2 <module'sub.mod2' from'../../LP6E/Chapter24/ns/part2/sub/mod2.py'> 即使我们立刻就通过命名空间包的名字导入更深层内容，这个跨文件夹的包也照样工作——因为命名空间包在首次触达时就已生成，路径扩展的时机无关紧要：$ python3 >>> import sub.mod1 Loading ns/part1/sub/mod1 >>> import sub.mod2 # 一个包横跨两个目录Loading ns/part2/sub/mod2 >>> sub.mod1 <module'sub.mod1' from'../../LP6E/Chapter24/ns/part1/sub/mod1.py'> >>> sub.mod2 <module'sub.mod2' from'../../LP6E/Chapter24/ns/part2/sub/mod2.py'> >>> sub <module'sub' (namespace) from ['../../LP6E/Chapter24/ns/part1/sub','../../LP6E/Chapter24/ns/part2/sub']> >>> sub.path NamespacePath(['../../LP6E/Chapter24/ns/part1/sub','../../LP6E/Chapter24/ns/part2/sub']) 有趣的是，如果我们按示例 24-12 加上一句，相对导入在命名空间包中同样有效。示例 24-12. ns/part1/sub/mod1.py（修改版）python from . import mod2 print('Loading ns/part1/sub/mod1') 这句新增的包相对导入语句引用的是包中的一个文件——尽管被引用的文件位于不同的目录：$ python3 >>> import sub.mod1 # 相对导入另一个目录中的 mod2 Loading ns/part2/s

```

ub/mod2 Loading ns/part1/sub/mod1 >>> import sub.mod2 # 已被 mod1.py 导入: 不会重跑>>> sub.mod2 <module'sub.mod2' from'../../LP6E/Chapter24/ns/part2/sub/mod2.py'> 如你所见, 除了物理存储被拆分之外, 命名空间包在各方面都与普通的单目录包一样。由此推论: 没有 init.py 文件的代码文件夹与常规包完全相同, 只是没有需要运行的初始化逻辑——它们不过是单目录的命名空间包实例而已。和包相对导入一样, 本书把这一主题的更多细节也留给 Python 手册。命名空间包是一个有潜在用处的工具, 但大多数 Python 学习者最好先掌握与真实文件夹对应的包, 再去攻坚那些虚拟地散布在宿主各目录之间的包。

深度理解

- 核心概念: 把两个互不相干的目录 (part1/sub 与 part2/sub) 通过 PYTHONPATH 合并成一个逻辑包 sub——这就是“虚拟拼接”的全部含义。
- 底层实现: import sub 时, 算法在候选集 (ns/part1、ns/part2) 中第 3 步记录了两个 sub 目录, 无更优命中, 于是创建命名空间包: file 缺失、path 是 Namespace Path(['ns/part1/sub','ns/part2/sub'])。之后 import sub.mod2 在 path 的两个目录中搜索, 命中第二个。
- 设计原因: 允许“一个名字, 多个物理存放点”——这对把大包拆成多个可独立安装的分发单元至关重要; from . import mod2 能跨目录找到兄弟, 正是因为相对导入的候选集是整个 path 列表, 而非单一目录。

·实际问题：大型框架（如 `setuptools` 生态）用命名空间包让"插件目录"与"核心目录"合并为同一包名；普通用户则受益于"任何目录都可导入"。

·初学者误区：以为"同名目录必须合并"或"命名空间包没有缓存"——它们照样进 `sys.modules`, `import sub.mod2` 第二次是 no-op（示例 24-12 演示了这一点）；也以为命名空间包"高级所以更好"——优先学常规包才是正道，Lutz 明确建议先掌握真实文件夹包。

代码分析

```
# ns/part1/sub/mod1.py — 24-12
from . import mod2          # ""= sub =
print('Loading ns/part1/sub/mod1')
```

解释器如何处理：`import sub.mod1` 时先创建 `sub` 命名空间包（`path` = 两个 `sub` 目录）；加载 `mod1.py` 时遇到 `from . import mod2`，相对导入把查找范围定为 `sub.path`（两个目录）：第一个目录（`part1/sub`）中没有 `mod2`，算法继续在第二个目录（`part2/sub`）中找到 `mod2.py` 并加载——于是先打印 `Loading ns/part2/sub/mod2`，再打印 `Loading ns/part1/sub/mod1`。随后 `import sub.mod2` 因缓存已存在而静默返回。这一行代码完美演示了：相对导入的"本包"可以是跨目录的虚拟集合。

24.26 本章小结 (Chapter Summary)

英文原文

This chapter introduced Python's package import model—an optional but useful way to explicitly list part of the directory path leading up to modules. Package imports are still relative to a directory on your module search path, but your script gives the rest of the path to the module explicitly. Packages may be built with a simple folder and can make imports more meaningful, simplify import search-path settings, and resolve ambiguities when there is more than one module of the same name—the name of the enclosing directory makes them unique. Because it's relevant only to code in packages, we also explored relative imports: a way for imports in package files to select modules in the same package explicitly using leading dots in `from`, instead of relying on a search in the host. Finally, we surveyed namespace packages: a tool that allows a package to span multiple directories as a fallback option of import searches, and make initialization files optional in single-folder packages. The next chapter surveys a handful of both common and advanced module-related topics, such as the `name` usage mode variable, the `getattr` attribute hook, and `name-string` imports, and codes useful module tools along the way. As usual, though, let's close out this chapter first with a short quiz to review what you've learned here.

中文翻译

本章介绍了 Python 的包导入模型——一种可选但有用的方式，用于显式列出通向模块的部分目录路径。包导入仍然相对于模块搜索路径上的某个目录，但你的脚本显式给出了通向模块的其余路径。包可以用一个简单的文件夹构建，它能让导入更有意义、简化导入搜索路径的设置，并在存在多个同名模块时消除歧义——外层目录的名字使它们变得唯一。因为只与包内代码相关，我们还探讨了相对导入：包文件中的导入用 `from` 前导点号显式选择同一包内的模块，而不是依赖在宿主机上的搜索。最后，我们概览了命名空间包：作为导入搜索的后备选项，它允许一个包横跨多个目录，并让单文件夹包中的初始化文件变为可选。下一章将概述一批常见和进阶的模块相关主题，例如 `name` 使用模式变量、`getattr` 属性钩子、名字字符串导入，并顺带编写有用的模块工具。不过照例，让我们先用一个小测验来复习本章所学，再结束本章。

深度理解

- 核心概念：本章三件大事——包导入（目录变命名空间）、相对导入（点号限定查找范围）、命名空间包（多目录虚拟合并）。
- 底层实现：三者共享同一导入管线，差异全在“查找起点与候选集”：`sys.path` → 包 `path` → 命名空间包的多目录 `path`。
- 设计原因：全部改动都围绕一个目标：让“名字”在文件系统层面也唯一、可预测。

·实际问题：掌握这三件事，就能理解 `pip install` 之后包为何可导入、`python -m` 与直接运行的差异、以及大库为何可以拆包分发。

·初学者误区：把"包"当成神秘黑箱——它只是"文件夹 + 约定文件名 + 搜索算法"，本章已全部拆解。

24.27 检查你的知识：测验 (Test Your Knowledge: Quiz)

英文原文

1. What is the purpose of an `init.py` file in a module package directory? 2. How can you avoid repeating the full package path every time you reference a package's content? 3. Which directories require `init.py` files? 4. When must you use `import` instead of `from with` packages? 5. What is the difference between `from pkg import name` and `from . import name`? 6. What is a namespace package?

中文翻译

1. 模块包目录中 `init.py` 文件的用途是什么？2. 如何避免每次引用包的内容时都重复完整的包路径？3. 哪些目录需要 `init.py` 文件？4. 什么时候对包必须用 `import` 而不是 `from`？5. `from pkg import name` 与 `from . import name` 有什么区别？6. 什么是命名空间包？

深度理解

- 核心概念：这六个问题是对本章知识点的精炼索引：init.py 的职责、路径缩短手段、包文件的历史沿革、命名冲突时的选择、两种导入语法的语义差异、命名空间包的定义。
 - 底层实现：每个问题的答案都落在导入机制的具体行为上（缓存、优先级、查找起点、path），而不是语法表面。
 - 设计原因：测验题型刻意包含一道"陷阱题"（问题 3）——提醒读者"曾经必须、现在可选"的规则变化，避免把过时经验当铁律。
 - 实际问题：这六问也是日常排查导入错误的排查清单。
 - 初学者误区：凭记忆答"所有目录都必须有 init.py"——这是 Python 3.2 之前的答案。
-

24.28 检查你的知识：答案 (Test Your Knowledge: Answers)

英文原文

1. The init.py file serves to declare and initialize a regular module package; Python automatically runs its code the first time you import through a directory in a process. Its assigned variables become the attributes of the module object created in memory to correspond to that directory. It's optional for package folders but gives a folder search precedence over other folders.

rs of the same name that don't have this file. 2. Use the from statement with a package to copy names out of the package directly, or use the as extension with the import statement to rename the path to a shorter synonym. In both cases, the path is listed in only one place, in the from or import statement. 3. Trick question! These files used to be required for packages in earlier Pythons but became optional with accommodations for namespace packages in the module search algorithm. As noted in answer 1, though, these folders still have valid, if optional, roles, including boosting a folder's import-search precedence. 4. You must use import instead of from with packages only if you need to access the same name defined in more than one path. With import, the path makes the references unique, but from allows only one version of any given name (unless you also use the as extension to rename uniquely). 5. The from pkg import name is an absolute import—the search for pkg skips an enclosing package and then tries the "absolute" directories in sys.path. A statement from . import name, on the other hand, is a relative import—name is looked up relative to the package in which this statement is contained, only. 6. A namespace package is an extension to the import model that corresponds to one or more directories that do not have init.py files. When Python finds these during an import search and does not find a simple module or regular package first, it creates a names

pace package that is the virtual concatenation of all found directories having the requested module name. Further nested components are looked up in all the namespace package's directories. The effect is similar to a regular package, but content may be split across multiple directories.

中文翻译

1. `init.py` 文件用于声明并初始化一个常规模块包；在进程中首次"经由"某目录导入时，Python 会自动运行它的代码。其中赋值的变量会成为内存中为对应目录创建的模块对象的属性。它对包文件夹是可选的，但能让该文件夹在搜索中优先于没有此文件的其他同名文件夹。2. 对包使用 `from` 语句，把名字直接从包中复制出来；或者把 `as` 扩展与 `import` 语句搭配使用，把路径改名为更短的别名。两种方式下，路径都只出现在一处——`from` 或 `import` 语句里。3. 陷阱题！在早期 Python 版本中这些文件对包是必需的，但随着模块搜索算法为命名空间包做出的调整，它们变成了可选的。不过如答案 1 所述，这些文件夹仍有一些正当（尽管可选）的作用，包括提升文件夹的导入搜索优先级。4. 只有当你需要在多个路径中访问同一个名字（定义在不止一条路径上的同名对象）时，才必须对包用 `import` 而不是 `from`。用 `import` 时，路径使引用唯一；而 `from` 只允许任意给定名字有一个版本（除非你也用 `as` 扩展做唯一重命名）。5. `from pkg import name` 是绝对导入——对 `pkg` 的搜索会跳过所在包，然后尝试 `sys.path` 中的"绝对"目录。而 `from . import name` 是相对导入——`name` 只相对于包含此语句的包来查找。6. 命名空间包是导入模型的一种扩展

，对应一个或多个没有 `init.py` 文件的目录。当 Python 在导入搜索中发现它们、且没有先找到简单模块或常规包时，它会创建一个命名空间包——即所有找到的、拥有请求模块名的目录的虚拟拼接。更深的嵌套组件会在命名空间包的所有目录中查找。效果与常规包类似，但内容可能横跨多个目录。

深度理解

- 核心概念：答案 5 是本章最重要的区分：`from pkg import name`（绝对，跳过所在包，查 `sys.path`）vs `from . import name`（相对，只查所在包）。记住这句，包内导入的成败一目了然。
- 底层实现：答案 6 精确描述了算法第 3 步与“无更优命中才创建”的后备语义，以及“虚拟拼接”后所有目录都参与更深层查找的机制。
- 设计原因：测验把“旧规则 vs 新规则”摆上台面，正是为了打破初学者从旧教程里带来的“必须有 `init.py`”的执念。
- 实际问题：答案 2 与答案 4 合起来，就是“路径太啰嗦怎么办”与“同名冲突怎么办”的完整对策：`from` 复制、`as` 别名、`import` 全路径。
- 初学者误区：把“`import` 与 `from` 只是写法不同”当成全部——在包场景里它们是“唯一性”与“复制”两种语义的选择，细节决定成败。

本章总结

技术拓展 (Technical Expansion)

- 实际项目中的应用场景：
 - 把工具代码组织成 mycompany/utils/ 包，init.py 聚合导出常用 API，对外只需 from mycompany.utils import loadconfig。
 - CLI 工具用 main.py 提供命令行入口，python -m mytool 即可运行，同时保留 import mytool 的库用法——python -m http.server、python -m pip 都是这个模式。
 - 命名空间包用于“插件体系”：核心框架占一个目录、第三方插件装进另一个目录，双方都叫 framework.plugins，导入时自动合并（setuptools 的 findnamespacepackages 支持这种布局）。
 - 排查导入报错时的标准顺序：ModuleNotFoundError → 检查包根是否在 sys.path (print(sys.path))；ImportError: attempted relative import ... → 检查文件是否可以包外脚本方式直接运行；“not a package” → 检查是否忘了去掉 .py 扩展名。
 - 与其他语言的区别 (Java/C++)：

对比项	Python 包	Java 包	C++ 头文件/命名空间
物理形态	文件夹 + init.py (可选)	文件夹 + package 声明	头文件 + namespace 关键字
查找机制	运行时按 sys.path/path 动态搜		

编译期按 classp
ath 搜索

编译期按 includ
e 路径搜索

初始化钩子	init.py 自动执行	静态初始化块/ 单例模式	无（靠手工调用）
可否跨目录合并	可以（命名空间包）	不可	不可（仅命名空间内名字逻辑合并）
运行时动态性	可运行时改 sys.path 再导入	需自定义类加载器	无

- Python 发展历史背景：Python 2 时代的包必须带 `init.py`，且支持“隐式相对导入”（包内 `import mod` 先找自己）——这造成大量歧义 bug；Python 3 废除隐式相对导入（必须显式 `.`），PEP 328 确立显式相对导入，PEP 420（Python 3.3）引入命名空间包、让 `init.py` 变为可选。本章的“为什么”几乎都是这三段历史的回响。

- 高级开发者需要掌握的相关知识：

- `importlib.importmodule('dir1.dir2.mod')`：用字符串动态导入包路径的官方方式。

- `pkgutil.itermodules()` 与 `pkgutil.walkpackages()`：遍历包内子模块做自动发现（插件注册）。

- 理解 `sys.modules` 缓存的副作用：`from` 复制的名字不随 `reload` 更新；`reload` 不递归。

- 打包分发（`pyproject.toml` + `setuptools/hatchling`）：`init.py` 中可放 `version`，让包版本号可编程读取。

学习建议 (Learning Advice)

- 重要程度：★★★★☆ (4/5)。日常脚本阶段可暂缓，但一旦开始写多文件项目、用标准库/第三方库，包就是必修课；尤其是 `python -m` 与直接运行的行为差异，几乎每

个开发者迟早都会踩到。

·应该掌握到什么程度：

1. 必会：用点号路径导入包内模块；`from 包.模块 import 名`与 `as` 别名；理解 `init.py` 的自动执行与命名空间初始化；理解 python 目录 vs python -m 包的差异。

2. 建议会：包内文件用完整路径或相对导入的取舍；`main.py` 做 CLI 入口；`__all__` 控制 `from`。

3. 了解即可：命名空间包的虚拟拼接与搜索算法四步；`path` 的手工扩展（高级技巧）。

·后续应该学习哪些相关内容：

·下一章（第 25 章）进阶模块主题：`__name__` 模式变量、`__getattr__` 钩子、字符串导入，以及“传递式模块重载”工具——它会用本章的包路径知识。

·第 22 章回顾 `sys.path`、`PYTHONPATH`、`.pth` 文件的完整配置细节。

·动手实验：搭建本章的 `dir1/dir2` 目录，用 REPL 跑一遍全部示例；再搭 `ns/part1/part2` 目录验证命名空间包的“虚拟拼接”，亲手观察 `sub.path` 与 `(namespace)` 标记。

·官方文档：PEP 328（相对导入）、PEP 420（命名空间包）、`importlib` 参考文档。